Web Based ML Model Evaluator

Team 13 final report

PES University, Bengaluru | UE24CS341A Software Engineering | Semester V

Prepared by Bhavesh Velluru on 9 November 2026. Review version 1.0. Implementation baseline `3f11b12` (PR #33 merged). This report consolidates the submitted baseline and the implementation available at that commit. Release acceptance remains conditional on the outstanding validation below. A separate project ID was absent from the supplied cover sheet; Team 13 is used as the submission identifier.

1. Synopsis and ownership

Students, researchers and teaching assistants repeatedly write scripts to load trained models, validate CSV inputs, compute metrics and compare runs. This application supplies a common workflow with isolated inference, trusted scoring and exportable results. The stakeholders are ML engineering students, researchers and course evaluators.

Member	USN / SRN	Functional ownership
Atharva Anand	PES1UG24CS092 / PES1202401515	M1 authentication, roles, history and workspaces
Atharva Rakesh Bidre	PES1UG24CS095 / PES1202400624	M2 model/CSV ingestion, target selection and compatibility
Ayush M Shetty	PES1UG24CS103 / PES1202403695	M3 Celery lifecycle, sandbox and trusted metrics
Bhavesh Velluru	PES1UG24CS115 / PES1202400414	M4 dashboards, comparison and JSON/PDF export; final documentation

The scope supports already-trained scikit-learn pickle/joblib and ONNX models, tabular classification and regression, and CSV test data. Training, GPU inference, deep-learning frameworks and image/video/NLP pipelines are outside Version 1 scope. The approved proposal is Team13_Synopsis.pdf.

2. Requirements and validation baseline

The approved combined SRS/SAD/STP identifies 27 functional, 10 security and 11 non-functional requirements. Its requirement IDs are the primary references for this report. The shorter SRS is retained as earlier context.

Requirement group	Delivered behavior	Evidence entry point
FR-001..007	Registration, JWT rotation/logout, role checks, private history, workspaces	tests/test_auth_history_workspaces.py
FR-008..013	Upload checks, CSV limits, target selection, sandbox compatibility inspection	tests/test_ingestion.py; tests/test_ingestion_api.py
FR-014..021	Asynchronous lifecycle, task detection, trusted metrics, timeout, cleanup	tests/test_evaluation_jobs.py; tests/test_metric_engine.py
FR-022..027	Charts, guarded comparison, best values, JSON and PDF exports	tests/test_benchmarking.py
SEC-001..010	Authentication, object access, file validation, containment, audit and configuration checks	tests/test_tsmp21_security.py; tests/test_sandbox_docker.py; tests/test_tsmp21_sec009.py
NFR-001..011	Timing, concurrency, recovery, portability, retention and quality checks	scripts/performance/; tests/test_retention.py; docs/TSMP_19_20_VALIDATION.md

The complete chain is in the RTM. That matrix and the 6 November validation reports predate the React merge. Their missing-UI findings need a final retest and reconciliation during Part 3. A source file's presence alone does not verify a requirement.

3. Project plan and Agile delivery

The approved baseline described six two-week iterations. The delivery backlog grouped work into three delivery sprints. The course Part 2 dates were 19-23 October and 26-30 October, with optional spillover/testing on 2-6 November. Git history shows Sprint 2 and Sprint 3 merges continuing in November. The team should present the actual history and carry-over dates in the demo.

Delivery group	Stories	Available outcome
Sprint 1	MLE-01..08	Repository/CI, auth/history/workspaces and input validation
Sprint 2	MLE-09..16	Worker/sandbox, metrics, dashboards, comparison and reports; PRs #29 and #30
Sprint 3	MLE-17..22	React workflow, security matrix, performance/retention tools and validation records; PRs #31, #32 and #33
Final submission	MLE-23..24 and Part 3	Consolidated report, maintenance, demo, risks, retrospectives and final test reports

The module owners implement features, open linked PRs and review one another's changes. Pull requests and commits supply contribution evidence. Historical dates and individual authors should be read directly from GitHub, since author identity in a commit alone cannot prove who performed a review.

4. Architecture and design

React/Vite serves the user workflow through an Nginx frontend. Django REST Framework exposes versioned APIs. PostgreSQL stores users, workspaces, uploaded-file metadata, compatibility reports, evaluations and report references. Celery workers consume Redis jobs. A restricted Docker socket proxy permits creation of a hardened container per evaluation. ReportLab renders downloadable PDFs.

Inference receives only staged model and feature inputs. Ground-truth labels and trusted metric computation stay outside the sandbox. Each job has enforced CPU, memory, process, filesystem and network constraints, with a 120-second runtime limit and cleanup/reaping paths. The approved SAD contains component, deployment, class, sequence, activity and ER diagrams; M3 traceability connects execution design to code.

The principal API groups are `/api/v1/auth`, `/models`, `/datasets`, `/compatibility-reports`, `/evaluations`, `/comparisons` and `/workspaces`. Uploads are validated before queuing. A successful compatibility result is required to create an evaluation. The queue returns 202 and a status URL; completed results provide metrics/charts and report downloads. Error responses use sanitized codes and preserve ownership checks.

5. Implementation and product workflow

The React routes in `frontend/src/App.tsx` include registration, login, upload, evaluation results, history and workspace. The separate Django `/results` dashboard provides classification/regression charts, comparison and downloads. These are distinct browser surfaces; the final demo should show the transition explicitly.

1. Register or sign in, then upload a supported trained model and CSV.

2. Choose the target and inspect compatibility results. Invalid or mismatched inputs receive an explanation.

3. Submit a compatible pair, poll the queued/running status and open the completed evaluation.

4. Inspect classification metrics, confusion matrix and ROC curves, or regression errors and residual plots.

5. Compare 2-5 successful owned runs with the same dataset checksum and task. Export JSON/PDF and inspect history/workspaces as the role permits.

Comparison minimizes MAE/MSE/RMSE and maximizes other scored scalar metrics. Exact ties select the first submitted evaluation. Upload paths use UUIDs; stored checksums preserve provenance. History survives expiry of uploaded files.

6. Testing and current evidence

Evidence scope	Recorded outcome	Qualification
Sprint 2	273 tests passed; 96.17% backend coverage	Historical run in SPRINT_2_EVIDENCE.md
6 November validation	381 passed; 7 strict xfails; 97.34% coverage	Historical execution report; contains missing-UI findings
9 November merge verification at f16df83	385 passed; 9 Docker skips; 7 strict xfails; 97.34% backend line coverage	Local Python 3.11, Redis available, Docker daemon unavailable
Updated CI	Core, security, sandbox, Compose and cross-OS tooling checks	All captured checks succeeded, including the deployed performance workflow; see evidence index for exact runs
Branch coverage	Final measurement pending Part 3	No branch percentage claimed here

The seven expected failures retain known specification gaps concerning evaluator access, ADMIN role behavior and HTTPS settings. Expected failures and infrastructure skips require explicit treatment in the final test record. The merged React UI needs reruns of the original system cases and five-participant usability study. Windows tooling checks exercise scripts, while full portability needs the actual Compose deployment on the required hosts.

The Part 3 package will contain updated cases, unit results, system validation, code coverage and branch coverage in `docs/`. Use one final commit, expected/actual outcomes and reproducible evidence links. See test execution report and defect register.

7. Validation, limitations and open decisions

The 6 November report counted 41 verified, 5 partial, 1 unverified and 1 unimplemented requirement. This is a dated baseline, not the acceptance count for the merged build. The new frontend removes the earlier absence of source but still needs test evidence before the four failed UI cases and usability case can be closed.

Remaining decisions include evaluator access to workspace-member results (DEF-05), ADMIN role policy (DEF-06), deployed HTTPS (DEF-09), the SonarQube gate (DEF-10), and when shared uploads should be removed following evaluation deletion (DEF-12). DEF-07 needs retest against the new UI. Availability requires a defined observation window. Acceptance of deviations must be recorded by the team/course reviewer.

The project is ready for final validation and demonstration preparation. Release acceptance requires the STP conditions to be evaluated on the final build, including High-priority requirements, the overall pass rate, open High/Critical defects and quality/security gates.

8. Screenshots and demo evidence

The captioned demo evidence index links seven existing product captures, PRs, source documents and CI runs. Those captures show the earlier `/results` dashboard. They are dated evidence of that surface. New React workflow captures and the final recording remain team actions listed in that index.

9. Maintenance

The maintenance plan defines corrective, adaptive, perfective and preventive work, owner responsibilities, regression checks, backup/recovery, retention and rollback.

10. Outcomes and further work

The project supplies reusable evaluation and comparison for uploaded trained tabular models, with isolated inference and scoring outside the untrusted process. The team produced a requirements/design/test baseline, incremental implementation, PRs and automated evidence. Further work includes closing the recorded policy/deployment gaps, measuring usability and availability, maintaining dependency pins and reconciling the final validation record.

Appendices

Repository

GitHub issues

Backlog and ownership

CI and demo evidence

Risk register and retrospectives

The PDF edition appends the approved Synopsis and SRS/SAD/STP unchanged, followed by captioned historical dashboard captures. Those baseline pages retain their original dates and numbering. The dated implementation and validation qualifications in this report govern the current release assessment.

PES UNIVERSITY

Department of Computer Science and Engineering

PROJECT SYNOPSIS / PROPOSAL

Course: UE24CS341A - Software Engineering | Semester: V

Project Title	Web Based ML Model Evaluator
Group Number	Team 13
Development Methodology	Agile (Scrum / Iterative Lifecycle)

1. Team Profile and Functional Ownership

The project is divided into complete functional areas so that each team member has clear ownership of one part of the system. All members will also participate in common project activities such as integration, reviews and testing.

Member	USN / SRN	Owned Functional Feature	Secondary Duties
Atharva Anand	PES1UG24CS092 PES1202401515	**User Authentication and Workspace Management** Registration, JWT login, RBAC and evaluation history.	Django authentication views and PostgreSQL user schema.
Atharva Rakesh Bidre	PES1UG24CS095 PES1202400624	**Dataset Ingestion and Schema Pre-Validation** Upload of model/dataset files, format checks, size checks, schema compatibility and target detection.	React upload interface and media storage setup.
Ayush M Shetty	PES1UG24CS103 PES1202403695	**Sandboxed Model Execution and Metrics Engine** Isolated model execution, task routing and metric calculation.	Celery task queue and Docker sandbox policy.
Bhavesh Velluru	PES1UG24CS115 PES1202400414	**Model Benchmarking, Visual Analytics and Report Export** Comparison tables, charts and PDF/JSON report generation.	Recharts interface and ReportLab PDF generation.

2. Problem Statement

Evaluating trained machine learning models for different datasets or model configurations often involves writing separate Python scripts or notebook code for loading the model, preparing the data, calculating metrics and creating plots. This becomes repetitive when several models need to be compared. Existing experiment tracking tools can also require changes to the training pipeline or additional SDK integration.

The proposed system provides a web interface where a user can upload a trained model and a test CSV dataset. The system validates the files, runs the evaluation in an isolated environment, displays the results and allows the user to compare models and export the results as reports.

3. Target Users / Stakeholders

- **ML Engineering Students and Researchers:** users who need to evaluate already-trained models without repeatedly writing evaluation scripts.
- **Course Evaluators and Teaching Assistants:** users who may need to evaluate model submissions using a common evaluation process.

4. Objectives

- Provide a web-based interface for uploading trained models and test datasets.
- Validate file format, size and feature compatibility before model execution.

- Run model inference inside an isolated Docker environment.
- Automatically identify classification or regression evaluation and calculate the relevant metrics.
- Display useful evaluation plots and metric summaries.
- Allow multiple evaluated models to be compared on the same dataset.
- Provide downloadable PDF and JSON summaries of evaluation results.

5. Product Scope

In scope: classical machine learning models supported by scikit-learn and ONNX models, evaluated using tabular CSV datasets. The system covers model/dataset upload, validation, sandboxed execution, metric calculation, visualization, comparison and report export.

Out of scope for Version 1: model training, real-time streaming inference APIs, and unstructured media pipelines such as image, video and NLP data.

6. Functional Features

- User account registration, JWT authentication and workspace evaluation history.
- Model and dataset upload with format, size and schema validation.
- Sandboxed model execution with automatic classification/regression task detection.
- Metric calculation for classification and regression tasks.
- Interactive visual dashboard for evaluation results.
- Side-by-side benchmarking of multiple evaluated models.
- PDF and JSON summary report export.

7. Plan of Work

Sprint	Timeline	Planned Work / Evidence	Owner
1	Weeks 1–2	Project setup, Django REST backend, JWT authentication APIs and initial Jira backlog.	Atharva A.
2	Weeks 3–4	File ingestion, CSV schema validation and upload interface.	Atharva B.
3	Weeks 5–6	Docker sandbox worker, task execution and ML metric services.	Ayush S.
4	Weeks 7–8	Model comparison and ReportLab-based report generation.	Bhavesh V.
5	Weeks 9–10	React dashboard integration, charts and SonarQube review.	All members
6	Weeks 11–12	GitHub Actions CI/CD, security hardening, system demonstration and final report preparation.	All members

8. Proposed Technology and Tools

- **Frontend:** React.js (Vite), Tailwind CSS, Recharts / Plotly.js
- **Backend:** Python 3.11, Django, Django REST Framework, Celery and Redis
- **ML / Reports:** scikit-learn, NumPy, Pandas, ONNX Runtime and ReportLab
- **Development / Testing:** Docker, PostgreSQL 15, Git/GitHub, GitHub Actions, SonarQube and Postman

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

UE24CS341A – Software Engineering | Level 3 Mini-Project | Semester V

Web Based ML Model Evaluator

Project Documentation Submission

Software Requirements Specification (SRS)

Software Architecture & Design Specification (SAD)

Software Test Plan (STP)

Item	Entry
Group Number	Team 13
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile (Scrum / iterative lifecycle), six two-week sprints
Document Version	1.0
Date	October 2026
Customer / Stakeholders	ML engineering students and researchers; course evaluators and teaching assistants

Team member	USN / SRN	Owned functional feature
Atharva Anand	PES1UG24CS092 / PES1202401515	M1 User Authentication and Workspace Management
Atharva Rakesh Bidre	PES1UG24CS095 / PES1202400624	M2 Dataset Ingestion and Schema Pre-Validation
Ayush M Shetty	PES1UG24CS103 / PES1202403695	M3 Sandboxed Model Execution and Metrics Engine
Bhavesh Velluru	PES1UG24CS115 / PES1202400414	M4 Model Benchmarking, Visual Analytics and Report Export

Contents

PART A – SOFTWARE REQUIREMENTS SPECIFICATION (SRS) 4

1. Introduction 4
2. Overall Description 5
3. External Interface Requirements 6
4. Analysis Models 6
5. System Features 8
6. Non-Functional Requirements 10
7. Other Requirements 11

Appendix A – Glossary 12

Appendix B – Field layouts and report requirements 12

PART B – SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION (SAD) 13

1. Introduction 13
2. High-Level Architecture 13
3. Module Decomposition and Division of Work 16
4. UML Models 17
5. Detailed Design of Key Modules 21
6. Data Design 22
7. API Design 23
8. User Interface Design 24
9. Security Design 25
10. Design-to-Validation Update 25

PART C – SOFTWARE TEST PLAN (STP) 26

1. Introduction 26
2. Test Items and Scope 26
3. Test Strategy 26
4. Test Environment 27
5. Entry, Exit and Suspension Criteria 27
6. Roles and Responsibilities 28
7. Schedule 28
8. Risks and Mitigation 28
9. Test Cases 28
10. Test Data Matrix 34
11. Security Testing Plan 34
12. Defect Management 34
13. Requirement Traceability Matrix 35

14. Test Deliverables and Reporting 36

How to read this document. *Part A (SRS) states what the system must do, Part B (SAD) states how it is structured and designed, and Part C (STP) states how it will be verified. Requirement IDs (FR-, NFR-, SEC-), use case IDs (UC-), design references (ARCH-, API-) and test IDs (UT-, IT-, ST-) are used consistently in all three parts and are tied together by the Requirement Traceability Matrix in Part C, Section 13.*

PART A – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Web Based ML Model Evaluator | Version 1.0

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements of the **Web Based ML Model Evaluator**, a web application in which a user uploads an already-trained machine learning model and a test CSV dataset, and receives validated, reproducible evaluation metrics, visual analytics, side-by-side model comparison and downloadable reports. It is the baseline for design, implementation, testing and validation of the product.

1.2 Document conventions

- Every requirement has a unique tag: **FR-nnn** (functional), **NFR-nnn** (non-functional), **SEC-nnn** (security), **BR-nn** (business rule), **UC-nn** (use case).
- “Shall” denotes a mandatory requirement. Priority is High, Medium or Low (MoSCoW: Must / Should / Could).
- Each requirement is intended to be concise, complete, unambiguous, verifiable and necessary; each lists its verification method.

1.3 Intended audience

Team members (developers and testers), the course faculty and teaching assistants evaluating the submission, and future maintainers.

1.4 Product scope

The product removes the need to write a separate Python script or notebook every time a trained model must be evaluated or compared, and does not require changes to the training pipeline or any experiment-tracking SDK.

In scope (Version 1)	Out of scope (Version 1)
Classical ML models serialized with scikit-learn (.pkl, .joblib) and ONNX models (.onnx)	Model training or hyper-parameter tuning
Tabular CSV test datasets (classification and regression)	Image, video, text/NLP and other unstructured pipelines
Registration, JWT login, RBAC, workspaces and evaluation history	Real-time streaming or online inference APIs
Upload, format/size/schema validation and compatibility checks	Deep-learning frameworks (PyTorch, TensorFlow) and GPU execution
Sandboxed execution in Docker and metric calculation	Hosting or serving models for third parties
Interactive dashboard, multi-model comparison, PDF and JSON reports	Social login, payments, mobile native apps

1.5 References

- Team 13 Project Synopsis / Proposal, UE24CS341A, Semester V.
- PES University, Software Engineering Level 3 Mini-Project: Student Submission Guide & Complete Exemplar.
- IEEE Std 29148-2018, Systems and software engineering – Requirements engineering; IEEE Std 829-2008, Software test documentation.
- scikit-learn, ONNX Runtime, Django REST Framework, Celery, Docker Engine and OWASP ASVS (Level 1) documentation.

2. Overall Description

2.1 Product perspective

The product is a new, self-contained client–server system. A React single-page application talks to a Django REST API over HTTPS. Long-running evaluation work is delegated through Redis to Celery workers, which run the uploaded model inside a short-lived Docker container. Results and metadata are stored in PostgreSQL; uploaded files and generated reports are kept on a media volume (see the architecture in Part B).

2.2 Product functions and ownership

Module	Functional area	Summary	Owner
M1	Authentication and Workspace Management	Registration, JWT login/refresh/logout, RBAC, workspaces, evaluation history	Atharva Anand
M2	Dataset Ingestion and Schema Pre-Validation	Model/dataset upload, format and size checks, target detection, schema compatibility	Atharva Rakesh Bidre
M3	Sandboxed Model Execution and Metrics Engine	Asynchronous jobs, task routing, isolated execution, classification/regression metrics	Ayush M Shetty
M4	Benchmarking, Visual Analytics and Report Export	Dashboard, model comparison, PDF and JSON reports	Bhavesh Velluru

Each member owns one complete, end-to-end functional area; UI work, database interaction and testing are secondary duties (React upload interface, Django authentication views, Celery/Docker policy, ReportLab/Recharts) and do not substitute for ownership.

2.3 User classes and characteristics

User class	Characteristics	Privileges
User (ML student / researcher)	Technically capable; evaluates own trained models; wants fast, repeatable evaluation	Register, upload own files, run and compare own evaluations, export reports, manage own history
Evaluator (course evaluator / teaching assistant)	Evaluates many submissions with one common process	All User privileges plus create course workspaces, add users, view evaluations created inside those workspaces
Administrator	Trained operator; low frequency of use	Activate/deactivate users, view audit log and system status; cannot read model or dataset content
Docker Engine (external system)	Local container runtime	Creates, runs and removes sandbox containers on request of the worker

2.4 Operating environment

- Server: Linux Docker host (Docker Compose) running Python 3.11, Django, Django REST Framework, Celery, Redis and PostgreSQL 15.
- Client: current versions of Chrome, Firefox and Edge on desktop; minimum viewport 1280 x 720 (responsive layout is a Could-have).
- Sandbox image: python:3.11-slim with pinned scikit-learn, NumPy, Pandas, joblib and ONNX Runtime.

2.5 Design and implementation constraints

- Technology stack fixed by the synopsis: React (Vite) + Tailwind + Recharts/Plotly; Django + DRF + Celery + Redis; PostgreSQL; ReportLab; Docker; GitHub Actions; SonarQube; Postman.
- Uploaded model files are untrusted executable content (a pickle can run arbitrary code when loaded) and shall therefore only be loaded inside the sandbox.
- The team follows Agile/Scrum with GitHub and Jira, pull-request reviews and CI/CD; every member must implement and be able to explain their module.
- The Version 1 delivery window is six two-week sprints (12 weeks) with a four-person team.

2.6 Assumptions and dependencies

- Users supply a test dataset that contains the target (ground-truth) column and the same feature columns, in the same order, that the model was trained on.
- Models are self-contained: any pre-processing must be part of the serialized object (for example a scikit-learn Pipeline) or the ONNX graph.

- The serialized scikit-learn version is compatible with the version pinned in the sandbox image; a mismatch is reported as a validation or run failure.
- A Docker engine is available to the worker; the host has enough memory for the configured number of concurrent sandboxes.

3. External Interface Requirements

3.1 User interfaces

ID	Screen	Purpose
UI-01	Register / Login	Create account, authenticate, display validation errors
UI-02	New Evaluation (upload)	Drag-and-drop model and CSV, choose target column, see validation report
UI-03	Run Status	Live status of a job (QUEUED, RUNNING, SUCCEEDED, FAILED, TIMED_OUT) and error text
UI-04	History	Paginated, filterable evaluation list; open, delete, select for comparison
UI-05	Results Dashboard	Metric cards, confusion matrix, ROC, predicted-vs-actual, residuals, export buttons
UI-06	Compare	Side-by-side metric table with best-model highlighting and comparison charts
UI-07	Workspaces (Evaluator)	Create workspace, add members, browse member evaluations
UI-08	Admin console	User activation and audit log view

The interface shall use consistent navigation, show progress for uploads and jobs, and present every error as a short human-readable message.

3.2 Software interfaces

Interface	Used for	Contract
PostgreSQL 15	Users, workspaces, artifacts metadata, evaluations, audit log	Django ORM over TCP; credentials from environment
Redis	Celery message broker and result backend	Queue evaluations; at-least-once delivery with idempotent task
Docker Engine API	Create/start/wait/log/remove sandbox containers	Through docker-socket-proxy with an allow-list of API calls
scikit-learn / ONNX Runtime	Model loading and inference	Run only inside sandbox; predict, optional predict_proba
ReportLab	PDF report generation	In-process library call in worker

3.3 Communication interfaces and hardware interfaces

All client–server traffic uses HTTPS (HTTP in local development) with JSON bodies, except file uploads (multipart/form-data) and report downloads (application/pdf, application/json). Authentication uses an Authorization: Bearer header. The product has no direct hardware interfaces.

4. Analysis Models

4.1 Use case diagram

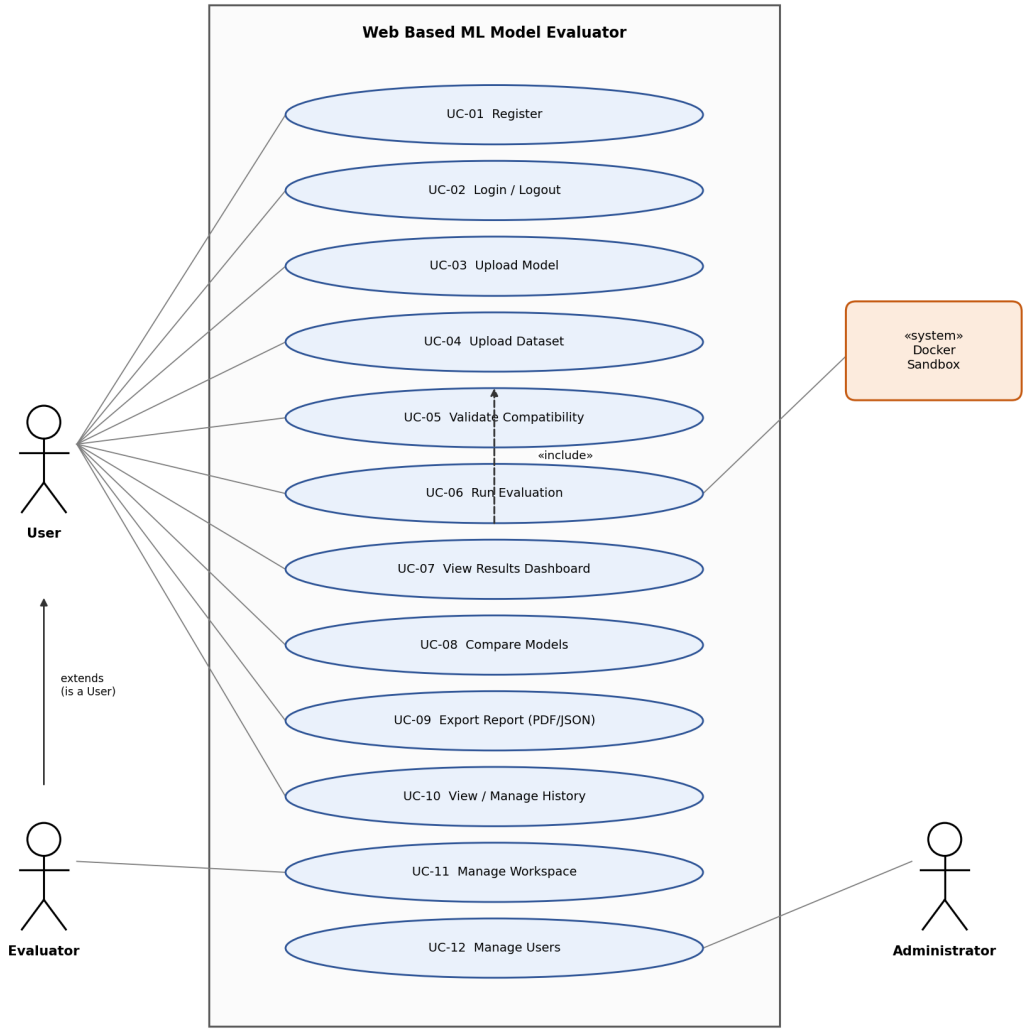


Figure 1: Use case diagram of the Web Based ML Model Evaluator

4.2 Use case descriptions

ID	Use case	Primary actor	Precondition	Main flow (summary)	Outcome
UC-01	Register	User	Application reachable	Enter email and password; system validates and creates account	Account created; duplicate email rejected
UC-02	Login / Logout	User, Evaluator, Admin	Account exists	Submit credentials; receive tokens; later refresh or log out	Authenticated session or revoked tokens
UC-03	Upload Model	User	Logged in	Select .pkl/.joblib/.onnx; system checks type, size, signature; stores file	Model artifact stored with checksum
UC-04	Upload Dataset	User	Logged in	Select CSV; system parses and checks structure; user confirms target column	Dataset stored with schema summary
UC-05	Validate Compatibility	User	Model and dataset uploaded	System inspects model input spec in sandbox and compares with dataset features	PASSED, or FAILED with a detailed report
UC-06	Run Evaluation	User	Validation PASSED	Submit job; worker runs sandbox, computes metrics, stores results	Evaluation SUCCEEDED / FAILED / TIMED_OUT
UC-07	View Results Dashboard	User	Evaluation SUCCEEDED	Open evaluation; view metric cards and plots	Metrics and charts displayed
UC-08	Compare Models	User	At least two SUCCEEDED evaluations on one dataset	Select 2–5 evaluations; system builds comparison table	Best model per metric highlighted
UC-09	Export Report	User	Evaluation or comparison exists	Click PDF or JSON; system generates and returns file	File downloaded
UC-10	View / Manage History	User	Logged in	List, filter, open or delete evaluations	History updated
UC-11	Manage Workspace	Evaluator	Logged in as EVALUATOR	Create workspace; add users; browse their evaluations	Shared workspace available
UC-12	Manage Users	Administrator	Logged in as ADMIN	Activate or deactivate accounts; view audit log	Account state changed

4.3 Evaluation lifecycle model

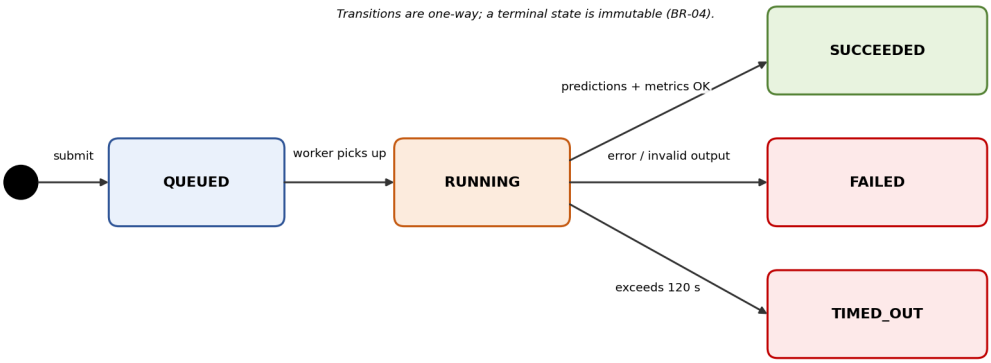


Figure 2: State model of an evaluation job

5. System Features

5.1 Authentication and Workspace Management (M1 – Atharva Anand)

Lets users register, sign in with JWT, and keep a private, filterable history of their evaluations. Evaluators can share course workspaces. Stimulus/response: a registration or login request returns tokens or an actionable error; a history request returns only data the caller may see.

ID	Requirement	Priority	Verification
FR-001	The system shall allow a visitor to register with a unique email address and a password of at least 8 characters containing letters and digits.	High	Functional + negative test

ID	Requirement	Priority	Verification
FR-002	The system shall authenticate a registered user and issue a signed JWT access token (15 min) and refresh token (7 days) carrying the user role.	High	Integration test
FR-003	The system shall renew access tokens using a valid refresh token (with rotation) and shall revoke the refresh token on logout.	High	Integration test
FR-004	The system shall enforce role-based access control (USER, EVALUATOR, ADMIN) on the server for every protected endpoint.	High	Authorization test
FR-005	The system shall show an authenticated user a paginated, filterable (status, task type, date) history of their own evaluations, newest first.	High	Integration + system test
FR-006	The system shall allow a user to delete their own evaluation record together with its stored artifacts and reports.	Medium	Integration test
FR-007	The system shall allow an EVALUATOR to create a course workspace, add users by email and view evaluations made inside that workspace.	Medium	Integration test

5.2 Dataset Ingestion and Schema Pre-Validation (M2 – Atharva Rakesh Bidre)

Accepts model and dataset files, rejects unsafe or malformed input before any model code runs, detects the target column and checks that model and dataset are compatible. Stimulus/response: each upload returns an identifier or a precise rejection reason; a validation request returns PASSED or a failure report.

ID	Requirement	Priority	Verification
FR-008	The system shall accept a model upload only if the extension is .pkl, .joblib or .onnx, the size is at most 100 MB and the content signature matches the extension.	High	Positive + negative test
FR-009	The system shall accept a dataset upload only if it is a CSV of at most 50 MB and 500,000 rows.	High	Boundary test
FR-010	The system shall validate dataset structure: header present, unique column names, at least one data row, and no missing values in feature columns.	High	Unit + negative test
FR-011	The system shall suggest a target column and allow the user to confirm or override it before evaluation.	Medium	Unit + functional test
FR-012	The system shall check model-dataset compatibility (number of features and, where the model exposes them, feature names) before an evaluation can be created.	High	Integration test
FR-013	The system shall return a validation report that names each failed check, the expected value and the actual value.	Medium	Functional test

5.3 Sandboxed Model Execution and Metrics Engine (M3 – Ayush M Shetty)

Runs inference in an isolated Docker container, keeps ground-truth labels outside the sandbox, detects the task type and computes metrics in trusted code. Stimulus/response: submitting a job returns immediately with a job id; the job moves through QUEUED, RUNNING and a terminal state with stored metrics or a sanitized error.

ID	Requirement	Priority	Verification
FR-014	The system shall create an evaluation job only for a validated model-dataset pair and process it asynchronously, returning the job id immediately.	High	Integration + system test
FR-015	The system shall automatically detect classification or regression from the target column and allow the user to override the detected task type.	High	Unit test
FR-016	The system shall execute model inference only inside an isolated Docker container (no network, non-root, read-only root filesystem, CPU/memory/PID limits).	High	Integration + security test
FR-017	The system shall compute accuracy, precision, recall, F1 (macro and weighted), confusion matrix and, where probabilities are available, ROC-AUC for classification.	High	Unit test
FR-018	The system shall compute MAE, MSE, RMSE and R-squared for regression.	High	Unit test
FR-019	The system shall track job status (QUEUED, RUNNING, SUCCEEDED, FAILED, TIMED_OUT) and terminate any job exceeding 120 seconds of execution.	High	Unit + integration test
FR-020	The system shall report a failed job with a sanitized message that contains no stack trace or server path.	Medium	Negative test
FR-021	The system shall persist the metrics and chart data of every successful evaluation and make them retrievable by evaluation id.	High	Integration test

5.4 Model Benchmarking, Visual Analytics and Report Export (M4 – Bhavesh Velluru)

Turns stored results into interactive charts, compares 2–5 models on one dataset and exports PDF and JSON reports. Stimulus/response: opening an evaluation renders its dashboard; selecting evaluations to compare returns an aligned table; export returns a file.

ID	Requirement	Priority	Verification
FR-022	The system shall display an interactive dashboard: metric cards, confusion matrix and ROC curve (classification) or predicted-vs-actual and residual plots (regression).	High	System test
FR-023	The system shall allow 2 to 5 completed evaluations to be compared side by side.	High	Unit + integration test
FR-024	The system shall present a comparison table that highlights the best model per metric (higher-is-better or lower-is-better as appropriate).	Medium	Unit + system test
FR-025	The system shall export the results of an evaluation or comparison as a JSON file containing metadata and metrics.	Medium	Integration test
FR-026	The system shall generate a PDF report for an evaluation or comparison containing dataset/model details, metric tables and charts.	High	Integration + system test
FR-027	The system shall reject a comparison of evaluations that use different datasets (checksum) or different task types.	High	Negative test

6. Non-Functional Requirements

6.1 Performance, reliability, usability, maintainability and portability

ID	Category	Measurable requirement	Verification
NFR-001	Performance	95% of non-evaluation API requests (login, history, status, chart data) shall respond within 500 ms with 50 concurrent users.	Load test (Locust)
NFR-002	Performance	An evaluation of a dataset up to 10,000 rows x 50 features with a model up to 20 MB shall finish within 60 s of the job starting (queue wait excluded).	Performance test
NFR-003	Scalability	The system shall run at least 5 evaluations concurrently by configuring the number of Celery workers, without cross-job interference.	Load test
NFR-004	Availability	The deployed system shall be available at least 99% of the defined demonstration/operating window, excluding planned maintenance.	Operational review
NFR-005	Usability	A first-time user shall complete a first evaluation (upload, validate, run, view) in no more than 5 primary screens.	Usability walkthrough
NFR-006	Reliability	A failed, crashed or timed-out job shall never leave an orphaned container or a job stuck in RUNNING for more than 5 minutes.	Failure-injection test
NFR-007	Maintainability	Business logic shall be separated into the four modules; backend line coverage shall be at least 80% and the SonarQube quality gate shall report no blocker or critical issues.	Code review + CI report
NFR-008	Portability	The system shall start with a single docker compose up on Linux, Windows (Docker Desktop) and macOS, and the UI shall work on current Chrome, Firefox and Edge.	Installation test
NFR-009	Testability	Validation, task-detection and metric services shall be testable without Docker or a database through injectable interfaces.	Unit tests in CI
NFR-010	Data retention	Uploaded models, datasets and reports shall be deleted automatically 30 days after upload unless the evaluation is deleted earlier.	Integration test
NFR-011	Performance	Validation of a 50 MB CSV shall complete within 20 seconds.	Performance test

6.2 Security requirements

ID	Security requirement	Validation approach
SEC-001	Passwords shall be stored only as salted hashes (Django PBKDF2/Argon2), never in plaintext or reversible form.	Code + database review
SEC-002	Role-based access control shall be enforced server-side; a USER token shall not reach EVALUATOR or ADMIN functions.	Authorization tests
SEC-003	Access tokens shall be signed, expire after 15 minutes, and expired, tampered or missing tokens shall be rejected with 401.	Security tests
SEC-004	All user-supplied input (file names, file content, query parameters, JSON fields) shall be validated and sanitized: allow-listed extensions, content-signature check, size limits, UUID storage names, parameterized queries.	Negative / security tests
SEC-005	Uploaded models are untrusted code. They shall run only in a sandbox with no network, non-root user, read-only root filesystem, dropped capabilities, no-new-privileges, CPU/memory/PID limits, a hard timeout, inputs mounted read-only, and without access to ground-truth labels.	Integration + security tests
SEC-006	Security-relevant events (failed login, authorization denial, sandbox policy violation, timeout) shall be written to an audit log.	System test
SEC-007	Login attempts shall be rate limited to 5 per minute per IP and account; uploads shall be rate limited per user.	Security test
SEC-008	Object-level authorization: a user shall access only their own artifacts, evaluations and reports (or those of a workspace they evaluate).	IDOR tests
SEC-009	Secrets shall be supplied through environment variables, never committed; production traffic shall use HTTPS and a restrictive CORS policy; dependencies shall be scanned.	Configuration review + scan
SEC-010	Error responses shall not disclose stack traces, file-system paths or internal identifiers.	Negative test

Threat note. *Python pickle and joblib files execute code on load. The principal risk of this product is therefore remote code execution through an uploaded model. The mitigation is architectural, not a file scan: untrusted models are loaded only in a locked-down container (SEC-005), and metrics are computed outside it so a hostile model can neither reach ground truth nor forge metrics beyond its own predictions.*

6.3 Business rules

ID	Rule
BR-01	Only a model–dataset pair whose validation status is PASSED may be evaluated.
BR-02	A comparison requires 2 to 5 SUCCEEDED evaluations that share the same dataset checksum and the same task type.
BR-03	Maximum sizes: model 100 MB, dataset 50 MB and 500,000 rows; execution time limit 120 s per job.
BR-04	An evaluation in a terminal state (SUCCEEDED, FAILED, TIMED_OUT) is immutable; re-running creates a new evaluation.
BR-05	Only the owner (or an EVALUATOR of the containing workspace) may view an evaluation and its reports.
BR-06	The target column is excluded from the features passed to the model and is never exposed to the sandbox.
BR-07	Datasets with missing values in feature columns are rejected in Version 1 (the report lists the affected columns).
BR-08	Only an ADMIN can deactivate users; deactivated users cannot log in but their audit history is retained.

7. Other Requirements

- **Data retention:** uploaded files and reports are removed 30 days after upload (NFR-010); audit logs are kept for 180 days.
- **Legal and privacy:** users upload only data they are entitled to use; the product stores no personal data other than email address and role. Datasets are not shared across users.
- **Localization:** English only, UTC timestamps displayed in the browser’s local time.
- **Deployment:** delivered as Docker Compose services (frontend, backend, worker, Redis, PostgreSQL, docker-socket-proxy) built and tested by GitHub Actions.
- **Reuse:** open-source libraries (scikit-learn, ONNX Runtime, Django, DRF, Celery, ReportLab, Recharts) are reused; sandbox policy, validation and metric orchestration are built by the team.

Appendix A – Glossary

Term	Meaning
Model artifact	A serialized, already-trained model file (.pkl, .joblib, .onnx)
Dataset	A tabular CSV test set containing feature columns and a target column
Target column	The ground-truth column the model should predict
Evaluation	One run of one model on one dataset, with its status, metrics and charts
Sandbox	A short-lived Docker container with restricted privileges in which untrusted model code runs
RBAC	Role-based access control
JWT	JSON Web Token used as a signed, expiring access credential
ROC-AUC	Area under the receiver-operating-characteristic curve
MAE / MSE / RMSE / R-squared	Mean absolute error, mean squared error, root mean squared error, coefficient of determination
Idempotent task	A task that gives the same result and side effects when delivered more than once

Appendix B – Field layouts and report requirements

B.1 Metrics reported

Task	Metrics and artifacts
Classification	Accuracy; precision, recall, F1 (macro, weighted and per class); confusion matrix; ROC curve and ROC-AUC when probabilities are available; class support
Regression	MAE, MSE, RMSE, R-squared; predicted-vs-actual scatter; residual plot and residual histogram

B.2 Report content (PDF and JSON)

- Header: report type, generated-at, evaluation id(s), owner.
- Model details: file name, format, size, SHA-256.
- Dataset details: file name, rows, columns, target column, SHA-256, task type.
- Metric table(s) and charts; for comparisons, one column per model with best values marked.
- Execution details: status, duration, sandbox image tag. The JSON export contains the same metadata and metrics in a documented schema.

Detailed database field layouts (names, lengths, types, mandatory flags) are given in the Software Architecture & Design Specification, Section 6. The Requirement Traceability Matrix (requirement → architecture → design → code → unit/integration test → system test) is given in Part C, Section 13.

PART B – SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION (SAD)

Web Based ML Model Evaluator | Version 1.0

1. Introduction

This document shows how the approved requirements in Part A become an implementable software structure: the high-level architecture, module decomposition and division of work, UML models, API contracts, data design, UI design, security design and the update to the validation specification.

1.1 Architectural drivers

Driver	Source	Architectural consequence
Untrusted uploaded models may run arbitrary code	SEC-005, FR-016	Dedicated sandbox container per job; labels and metrics kept outside; hardened run policy
Evaluations can take seconds to minutes	FR-014, NFR-002, NFR-003	Asynchronous job queue (Celery + Redis); API returns 202 and the client polls
Four members must work in parallel	Course guidelines	Four loosely coupled Django apps with stable REST contracts and one owner each
Reproducible, comparable results	FR-023, FR-027, BR-02	Dataset/model SHA-256 recorded; results stored as immutable JSON on the evaluation
Easy installation and grading	NFR-008	Single Docker Compose deployment; the same images used by CI
Testability without infrastructure	NFR-009	Services behind interfaces: validators, metrics and comparison are pure Python; sandbox is injectable

1.2 Design goals

- Security first: treat every uploaded byte as hostile.
- Separation of concerns: API, business services, data access and sandbox are separate layers.
- Stable contracts: REST schemas are versioned (/api/v1) and not changed silently once other modules depend on them.
- Simplicity: monolithic Django backend plus one worker, not microservices.

2. High-Level Architecture

The system uses a **layered client–server architecture with an asynchronous worker and a sandboxed execution boundary**. Request flow: User → React UI → REST API → service layer → repository (ORM) → PostgreSQL / media volume; for evaluations: API → Redis queue → Celery worker → Docker sandbox → worker (metrics) → PostgreSQL.

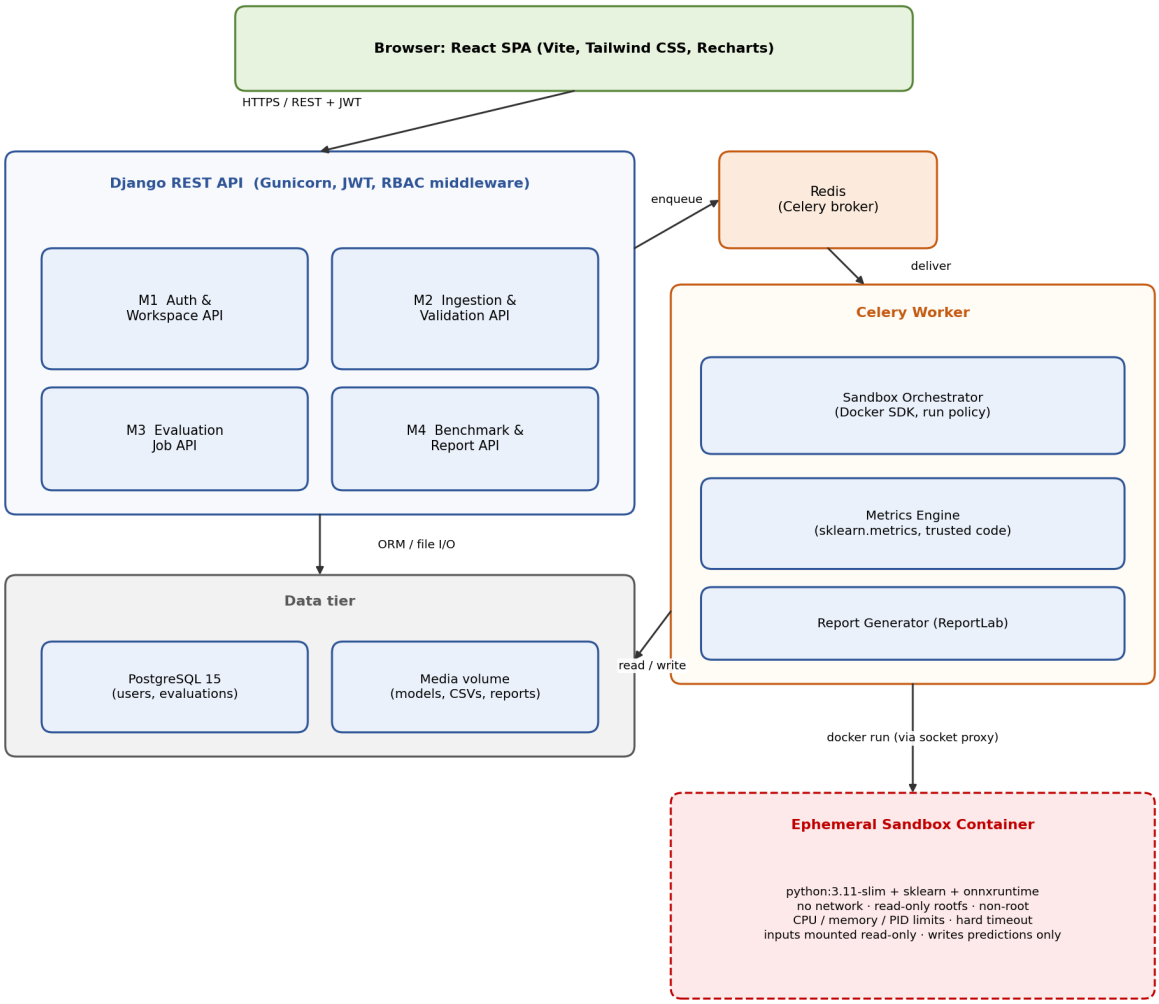


Figure legend: green = client, blue = API, orange = asynchronous processing, red = untrusted-code boundary

Figure 3: High-level architecture and component diagram

2.1 Logical layers

Layer	Components	Responsibility	Technology
Presentation	Auth pages, Upload wizard, Run status, History, Dashboard, Compare, Workspaces, Admin	Capture input, call REST API, render charts	React (Vite), Tailwind CSS, Recharts / Plotly.js
API / controller	accounts, ingestion, evaluations, benchmarking REST views and serializers	Routing, authentication, authorization, request validation	Django REST Framework, SimpleJWT
Business logic	AuthService, ValidationService, EvaluationService, SandboxRunner, MetricsEngine, ComparisonService, ReportGenerator	Business rules and workflows	Python 3.11
Data access	Django ORM models and repositories; storage adapter for media volume	Persistence and file I/O	Django ORM, django-storages (local)
Data	Users, Workspaces, ModelArtifacts, Datasets, Evaluations, Comparisons, Reports, AuditLog	Persistent data	PostgreSQL 15; media volume
Async processing	Celery tasks run_evaluation, generate_report, purge_expired	Long-running and scheduled work	Celery, Redis
External / runtime	Docker Engine (through socket proxy)	Run untrusted code in containers	Docker, docker-socket-proxy

2.2 Architecture reference IDs

Ref	Component	Requirements covered
ARCH-AUTH	Authentication and workspace service (M1)	FR-001..007, SEC-001..003, SEC-007, SEC-008
ARCH-ING	Ingestion and validation service (M2)	FR-008..013, SEC-004, NFR-011
ARCH-EXEC	Evaluation job service and metrics engine (M3)	FR-014..021, NFR-002, NFR-003, NFR-006
ARCH-SBX	Sandbox runner and container image (M3 with M2 for inspect mode)	FR-012, FR-016, SEC-005
ARCH-BENCH	Comparison, analytics and report service (M4)	FR-022..027
ARCH-UI	React single-page application (shared; each member builds the screens of their module)	UI-01..UI-08, NFR-005

2.3 Deployment view

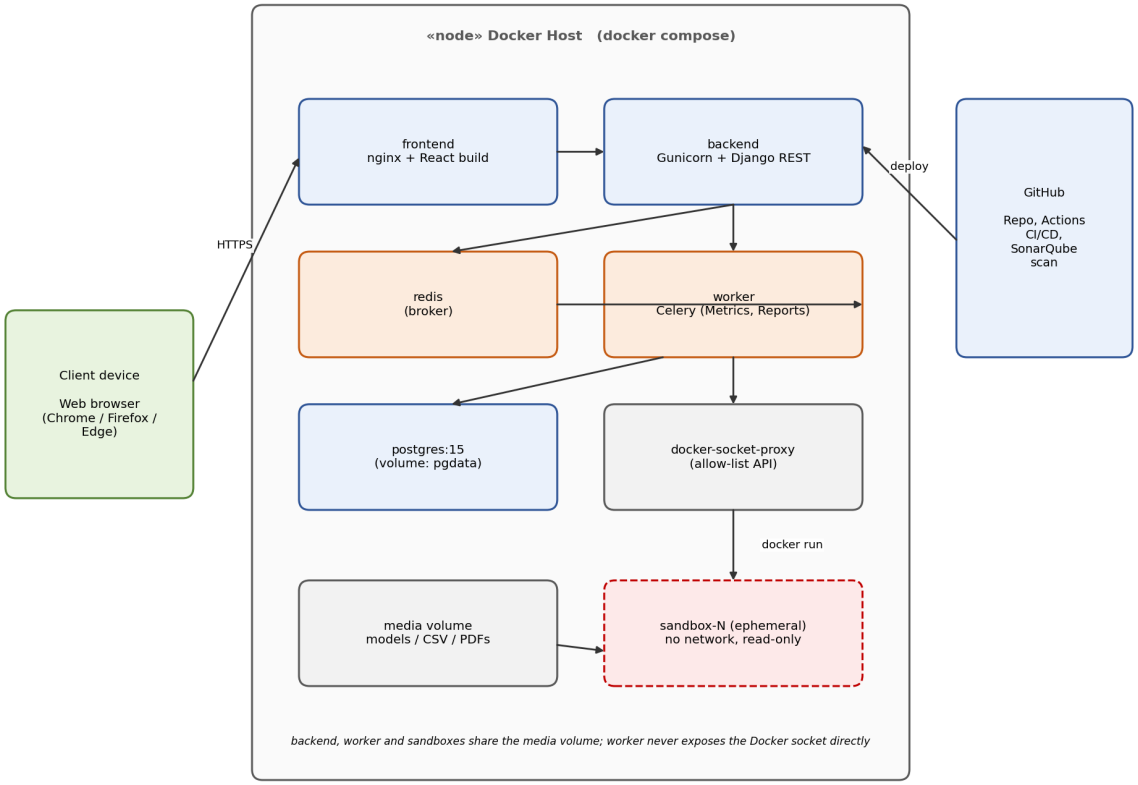


Figure 4: Deployment diagram (Docker Compose on one host, CI/CD from GitHub)

Backend, worker and sandbox containers share a media volume; sandboxes mount only the specific input files, read-only. The worker never receives the raw Docker socket: it talks to a socket proxy that allows only container create, start, wait, logs and remove, preventing a compromised worker from controlling the host.

3. Module Decomposition and Division of Work

Module	Django app / package	Responsibilities	Depends on	Owner
M1 Auth and Workspace	accounts/	Register, login, refresh/logout, RBAC permissions, workspaces, history, audit log	PostgreSQL	Atharva Anand
M2 Ingestion and Validation	ingestion/	Upload endpoints, file and CSV validators, target detection, compatibility check, media storage	accounts, storage, sandbox (inspect)	Atharva Rakesh Bidre
M3 Execution and Metrics	evaluations/ (tasks, sandbox, metrics)	Job API, Celery task, Docker sandbox runner, task detection, metric services, status machine	ingestion, Redis, Docker	Ayush M Shetty
M4 Benchmark and Reports	benchmarking/	Chart data, comparison, JSON export, ReportLab PDF, Recharts dashboard components	evaluations	Bhavesh Velluru

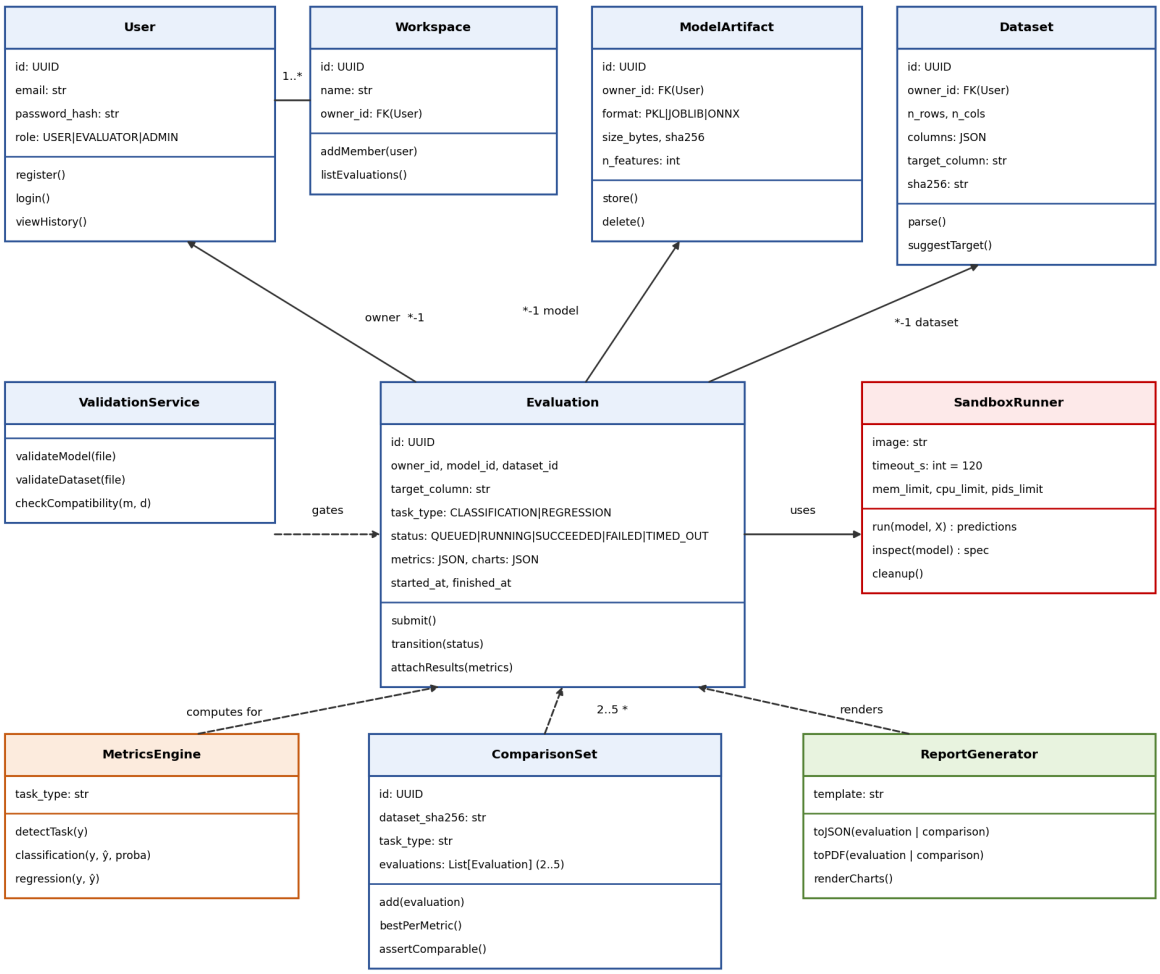
Secondary duties *(per synopsis): M1 – Django authentication views and PostgreSQL user schema; M2 – React upload interface and media storage setup; M3 – Celery task queue and Docker sandbox policy; M4 – Recharts interface and ReportLab PDF generation. All members take part in integration, code review and testing.*

3.1 Planned source layout

backend/ accounts/ ingestion/ evaluations/ (tasks.py, services.py, sandbox/runner.py, metrics/) benchmarking/ (comparison.py, exporters.py, reports/pdf.py) config/ · sandbox/ (Dockerfile, runner.py) · frontend/src/ (pages, components, api) · docker-compose.yml · .github/workflows/ci.yml · docs/

4. UML Models

4.1 Class diagram



Evaluation is the aggregate root: metrics, charts and status live on it; MetricsEngine, ComparisonSet and ReportGenerator only read or derive from it.

Figure 5: Class diagram of the major domain and service classes

4.2 Sequence diagram – Upload and Validate

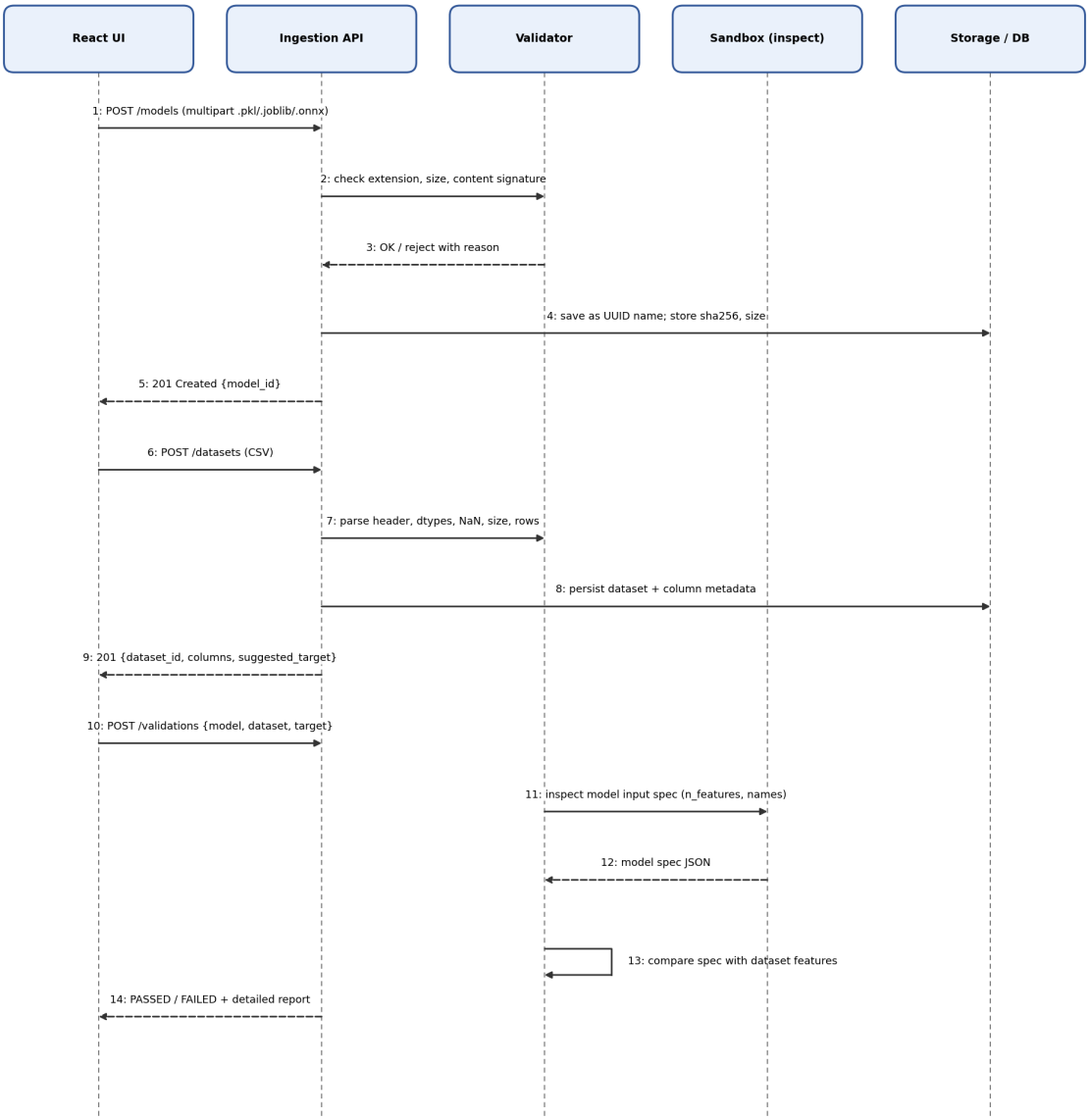


Figure 6: Sequence diagram: upload model and dataset, then validate compatibility

4.3 Sequence diagram – Run Evaluation (Process Order equivalent)

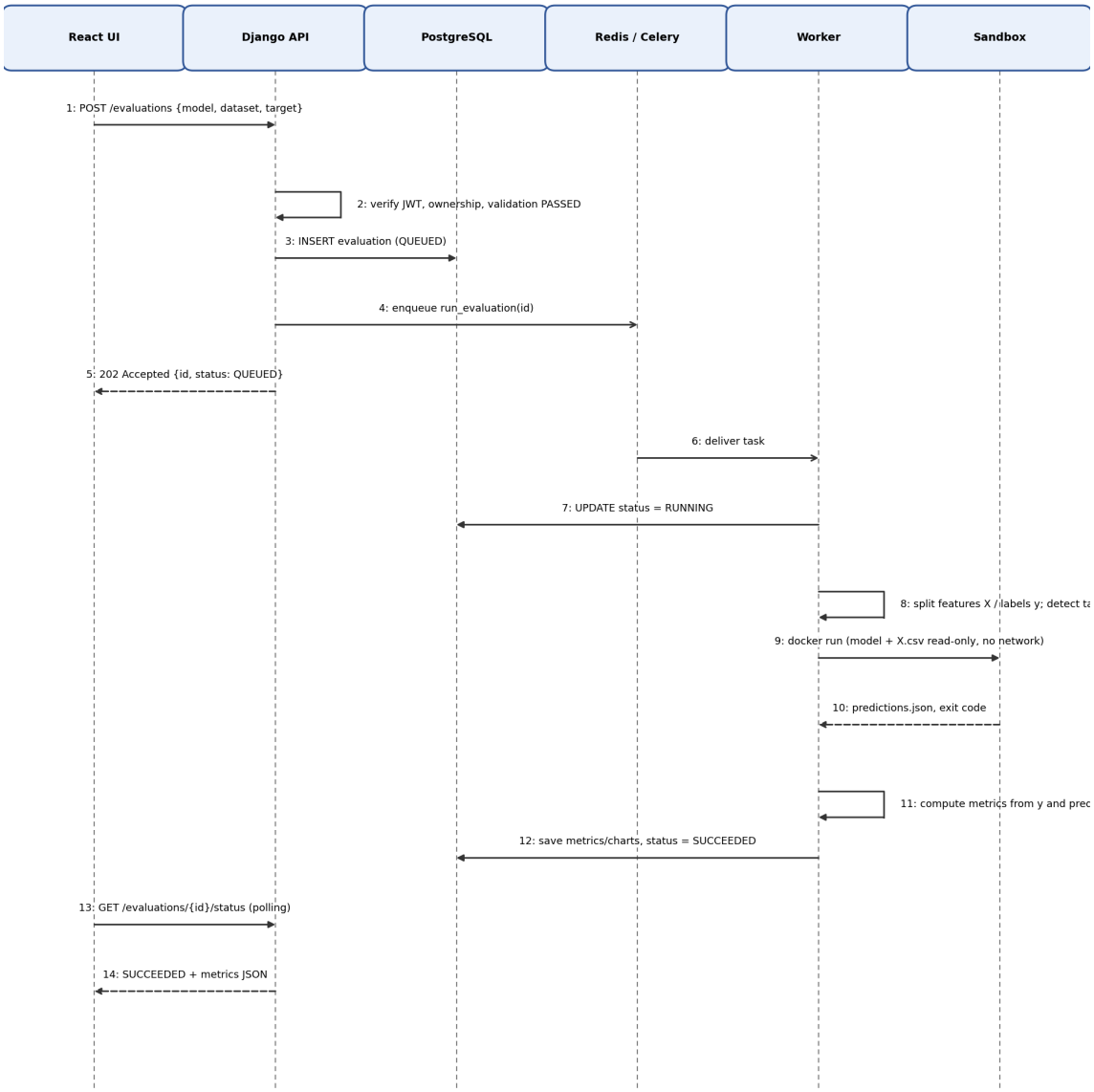


Figure 7: Sequence diagram: asynchronous evaluation through the sandbox

4.4 Activity diagram – evaluation workflow

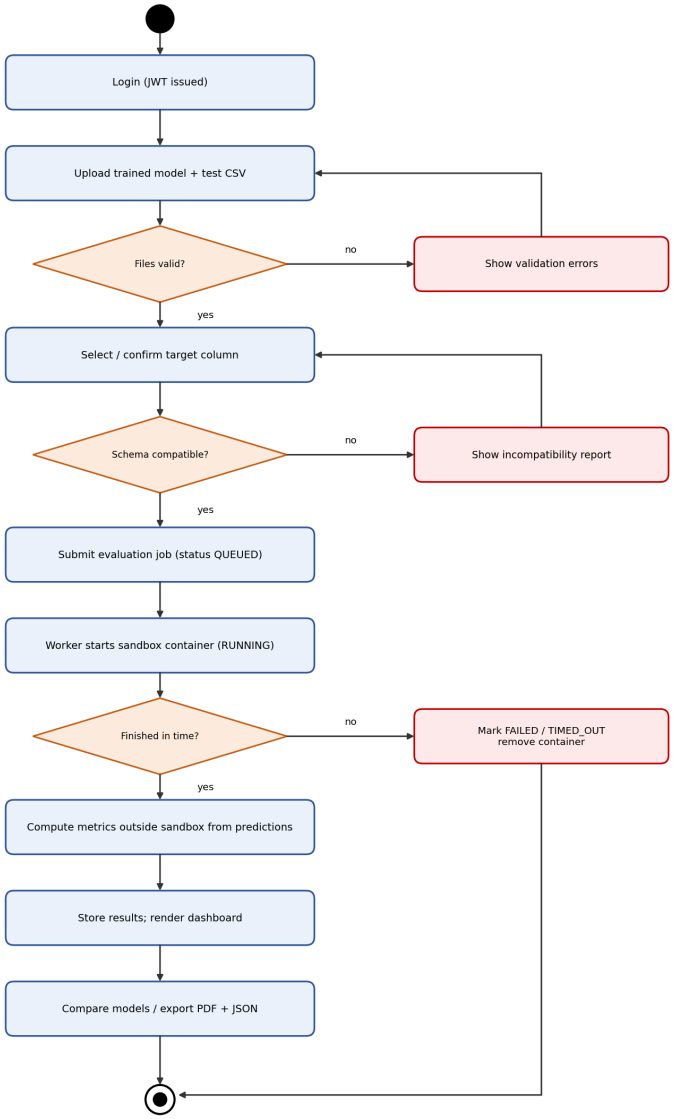


Figure 8: Activity diagram of the evaluation workflow with failure branches

The component and deployment diagrams are Figures 3 and 4 (Section 2); the state model of an evaluation is in Part A, Section 4.3.

5. Detailed Design of Key Modules

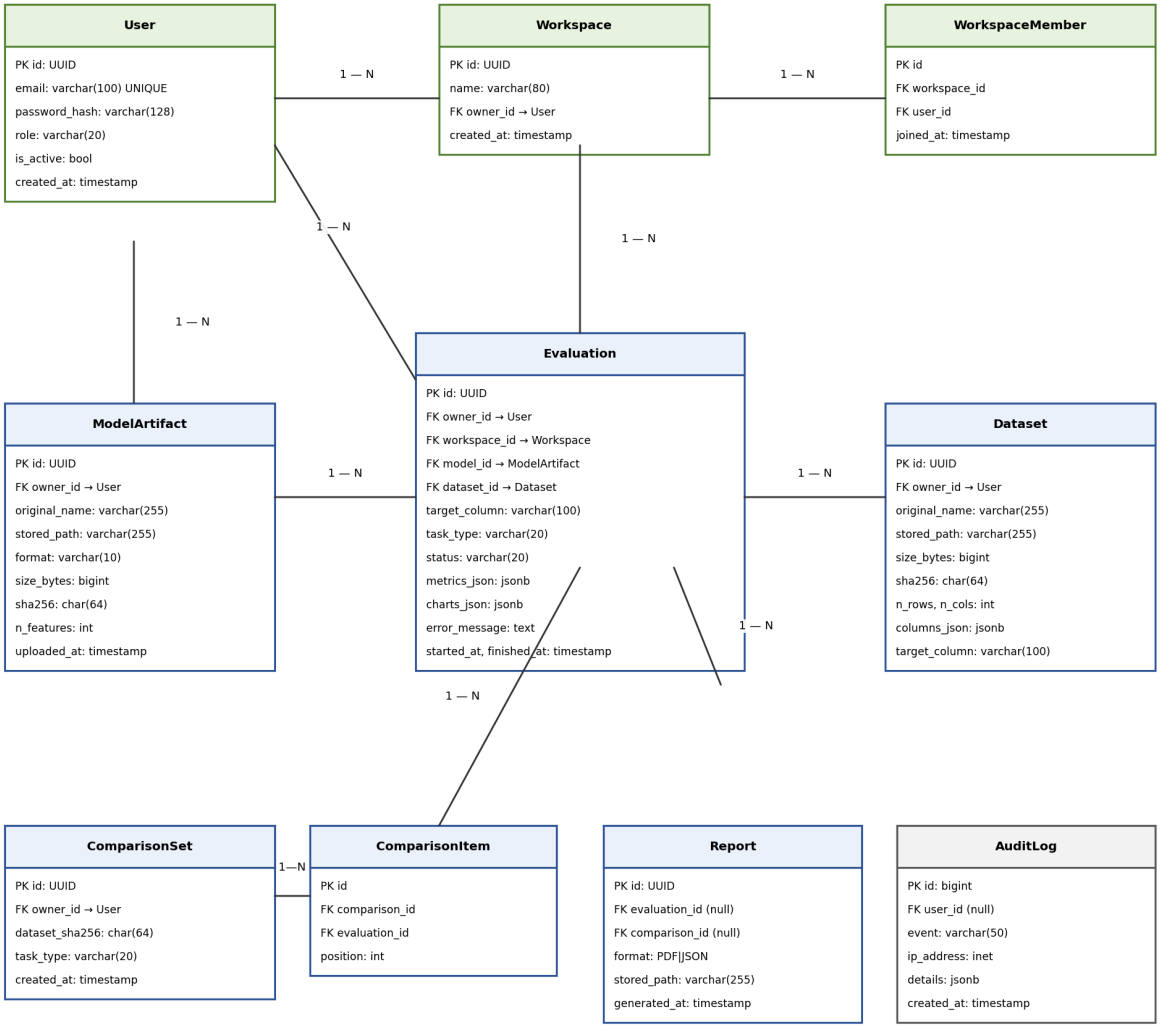
5.1 Sandbox execution design (M3)

Step	Design decision	Reason
1	Worker splits the CSV into **features X** and **labels y**; only X and the model file are mounted into the container (read-only).	A hostile model never sees ground truth (BR-06).
2	Container runs runner.py: loads model, runs predict (and predict_proba if present), writes predictions.json to a dedicated writable output directory.	Only predictions leave the sandbox.
3	Run policy: network none, read-only root FS, user 10001, cap-drop ALL, no-new-privileges, default seccomp, memory 1 GB, 1 CPU, PID limit 128, tmpfs /tmp 64 MB, timeout 120 s, auto-remove.	SEC-005; limits blast radius of hostile code.
4	Worker validates predictions.json (length, dtype, finite values) and computes metrics with trusted sklearn.metrics code.	A model cannot forge metrics beyond its own predictions.
5	On timeout the worker kills and removes the container, sets TIMED_OUT and writes an audit entry; on any error it sets FAILED with a sanitized message.	FR-019, FR-020, NFR-006.
6	Compatibility inspection (M2) uses the same sandbox in inspect mode for pickles (reads n_features_in_ / feature_names_in_); ONNX input shapes are read with a safe parser.	Even metadata reads need unpickling, so they are sandboxed too.

5.2 Key design decisions

ID	Decision	Alternatives considered	Rationale
D-01	Celery + Redis for jobs	Run in the request thread; Django-Q	Non-blocking API; independent worker scaling; matches the planned stack
D-02	One Docker container per evaluation	Long-lived shared sandbox; subprocess with rlimits	Strong isolation and clean state per job; simple resource limits
D-03	Metrics outside the sandbox	Compute metrics in the container	Keeps labels out and prevents forged results
D-04	JSON columns for metrics and charts	Normalized metric tables	Metric sets differ by task; results are read-mostly and immutable
D-05	JWT access (15 min) + rotating refresh (7 days)	Server sessions	Stateless API for the SPA; short exposure window; revocation via refresh blacklist
D-06	Polling for job status (2 s interval)	WebSockets	Simple, robust, adequate for Version 1; can be upgraded without API change
D-07	ReportLab for PDF, Recharts for UI charts, matplotlib images embedded in PDF	Headless-browser PDF	No browser dependency in the worker; deterministic output

6. Data Design



Every foreign key uses ON DELETE protection: deleting an Evaluation cascades to Report and ComparisonItem; Users are soft-deactivated (is_active) rather than deleted.

Figure 9: Entity-relationship diagram

6.1 Field layouts (principal tables)

Table.field	Length	Data type	Description	Mandatory
user.id	36	UUID	Unique user identifier (PK)	Y
user.email	100	varchar	Unique login email	Y
user.password_hash	128	varchar	Salted password hash	Y
user.role	20	enum	USER, EVALUATOR or ADMIN	Y
model_artifact.id	36	UUID	Model identifier (PK)	Y
model_artifact.stored_path	255	varchar	Path under media root (UUID file name)	Y
model_artifact.format	10	enum	PKL, JOBLIB or ONNX	Y
model_artifact.sha256	64	char	Checksum of uploaded file	Y
dataset.id	36	UUID	Dataset identifier (PK)	Y
dataset.columns_json	–	jsonb	Column names and inferred dtypes	Y
dataset.target_column	100	varchar	Confirmed target column	N (until confirmed)
dataset.sha256	64	char	Checksum used for comparison guard	Y
evaluation.id	36	UUID	Evaluation identifier (PK)	Y
evaluation.task_type	20	enum	CLASSIFICATION or REGRESSION	Y
evaluation.status	20	enum	QUEUED, RUNNING, SUCCEEDED, FAILED, TIMED_OUT	Y
evaluation.metrics_json	–	jsonb	Computed metrics (null until SUCCEEDED)	N
evaluation.error_message	–	text	Sanitized failure reason	N
comparison_set.dataset_sha256	64	char	Dataset shared by all items	Y
report.format	10	enum	PDF or JSON	Y
audit_log.event	50	varchar	Event name, for example LOGIN_FAILED	Y

6.2 Integrity constraints

- Unique: user.email; (workspace_id, user_id); (comparison_id, evaluation_id).
- Check: evaluation.status transitions enforced in the service layer; size and row limits enforced before persistence.
- Foreign keys protect referential integrity; deleting an evaluation cascades to its reports and comparison items; users are deactivated, not deleted.

7. API Design

Base path **/api/v1**. JSON request/response unless stated. Authentication column: **P** public, **U** any authenticated user (own data), **E** evaluator, **A** admin.

ID	Method	Endpoint	Purpose	Auth	Input	Output / status codes
API-01	POST	/auth/register	Create account	P	email, password	201 user id; 400 invalid; 409 duplicate
API-02	POST	/auth/login	Authenticate	P	email, password	200 access, refresh, role; 401; 429
API-03	POST	/auth/token/refresh	Rotate tokens	P	refresh	200 new tokens; 401
API-04	POST	/auth/logout	Revoke refresh token	U	refresh	204; 401
API-08	GET	/evaluations	History (paginated, filters)	U	status, task_type, from, to, page	200 list; 400; 401
API-09	GET	/evaluations/{id}	Evaluation detail with metrics	U	id	200; 401; 404
API-10	DELETE	/evaluations/{id}	Delete evaluation and artifacts	U	id	204; 401; 404
API-11	POST	/models	Upload model	U	multipart file	201 id, sha256; 400; 401; 413
API-12	POST	/datasets	Upload dataset	U	multipart CSV	201 id, columns, suggested_target; 400; 413

ID	Method	Endpoint	Purpose	Auth	Input	Output / status codes
API-13	PATCH	/datasets/{id}/target	Confirm / override target	U	column	200; 400; 404
API-14	POST	/validations	Check compatibility	U	model_id, dataset_id	200 PASSED/FAILED + report; 404
API-15	POST	/evaluations	Create job	U	model_id, dataset_id, task_type?	202 id, status QUEUED; 400 not validated; 404
API-16	GET	/evaluations/{id}/status	Job status	U	id	200 status, error_message; 404
API-17	GET	/evaluations/{id}/charts	Chart data	U	id	200 chart JSON; 409 not finished; 404
API-18	POST	/comparisons	Create comparison	U	evaluation_ids (2–5)	201 comparison; 400 guard failure; 404
API-19	GET	/evaluations/{id}/report.json	JSON export	U	id	200 JSON; 404
API-20	GET	/evaluations/{id}/report.pdf and /comparisons/{id}/report.pdf	PDF report	U	id	200 application/pdf; 404
API-21	POST	/workspaces	Create workspace	E	name	201; 403
API-22	POST	/workspaces/{id}/members	Add member by email	E	email	201; 403; 404
API-23	PATCH	/admin/users/{id}	Activate / deactivate user	A	is_active	200; 403; 404

7.1 API contract rules

- Request and response schemas are defined with DRF serializers and published as an OpenAPI document that is imported into the Postman collection.
- Errors use one schema: {"error": {"code": "VALIDATION_FAILED", "message": "...", "details": [...]}}; messages never include stack traces or paths.
- Status codes: 200/201/202/204 success; 400 validation; 401 unauthenticated; 403 forbidden role; 404 not found or not owned; 409 state conflict; 413 too large; 429 rate limited.
- Identifiers are UUIDs. Breaking changes require a new version prefix; additive changes only within /v1.
- Every endpoint declares its authentication and authorization requirement (see column Auth) and enforces it on the server.

8. User Interface Design

Screen	Key components	Interaction notes
Upload wizard (UI-02)	Two drop zones, file chips with size, target-column dropdown, Validate button, validation report panel	Three steps: Files, Target, Validate; error text shown next to the failing check
Run status (UI-03)	Status badge, elapsed time, spinner, error panel	Polls /status every 2 s; stops at terminal state
Dashboard (UI-05)	Metric cards, tabbed charts, download buttons	Classification and regression variants; charts via Recharts
Compare (UI-06)	Evaluation picker (2–5), metric table, grouped bar chart, best-value badges	Mismatched dataset or task disables Compare with an explanation
History (UI-04)	Table with status and task filters, pagination, delete, compare checkbox	Newest first; empty state links to the wizard

Primary path for a new user: Register → Login → New Evaluation → Dashboard (4 primary screens, satisfying NFR-005).

9. Security Design

Threat (STRIDE)	Asset / entry point	Mitigation	Req.
Spoofing	Login, tokens	Hashed passwords, short-lived signed JWT, rate limiting, refresh rotation	SEC-001, 003, 007
Tampering	Token payload, uploads, API input	Signature verification, allow-lists, content-signature checks, parameterized queries	SEC-003, 004
Repudiation	Security-relevant actions	Audit log with user, IP, event, time	SEC-006
Information disclosure	Other users' models, datasets, reports; error messages	Object-level authorization; 404 for non-owned objects; sanitized errors; labels never in sandbox	SEC-008, 010
Denial of service	Large uploads, runaway or resource-hungry models	Size/row limits, early 413, timeouts, memory/CPU/PID limits, queue per worker	SEC-005, NFR-006
Elevation of privilege	RBAC, uploaded model code, Docker socket	Server-side RBAC; sandbox without network/root/capabilities; socket proxy allow-list	SEC-002, 005

10. Design-to-Validation Update

Each significant design element raises a validation question that is answered by a planned test in Part C (this updates the initial validation specification).

Design element	Validation question	Planned test
Role-based API	Can a USER call evaluator or admin functions?	ST-SEC-02, IT-AUTH-12
Object-level authorization	Can user A read user B's evaluation, model or report?	ST-SEC-09, IT-REP-06
Upload validators	Can disguised, oversized or traversal-named files be stored?	IT-ING-02, ST-SEC-06, ST-SEC-11
Sandbox run policy	Does the container really have no network, no root, and limits?	IT-EXE-04, ST-SEC-07, ST-SEC-08
Labels kept outside sandbox	Is ground truth ever present in the container?	ST-EXE-02 (inspect mounts and environment)
Status state machine	Can illegal transitions occur?	UT-EXE-05
Job timeout and cleanup	Are runaway jobs killed and containers removed?	IT-EXE-05, REL-01
Comparison guard	Can different datasets or task types be compared?	UT-REP-02, IT-REP-02
Metrics service	Do metrics equal hand-computed values?	UT-EXE-03, UT-EXE-04
Database constraints	Can duplicate emails or orphan rows be created?	IT-AUTH-02, IT-AUTH-09

PART C – SOFTWARE TEST PLAN (STP)

Web Based ML Model Evaluator | Version 1.0

1. Introduction

1.1 Purpose and objectives

This plan defines how the Web Based ML Model Evaluator will be verified and validated against Part A. Objectives: show that every requirement is implemented (traceability), find defects early through automated tests in CI, demonstrate that the sandbox and access controls resist the threats in Part B, and produce the unit-test, coverage, security and validation evidence required for submission.

1.2 References

Part A (SRS), Part B (SAD), Team 13 Synopsis, Student Submission Guide; IEEE 829-2008; OWASP ASVS Level 1 and OWASP Web Security Testing Guide.

2. Test Items and Scope

Feature area (owner)	Items under test	Requirements
M1 Auth and Workspace (Atharva Anand)	Registration, login, refresh/logout, RBAC, history, workspaces	FR-001..007; SEC-001..003, 007, 008
M2 Ingestion and Validation (Atharva Rakesh Bidre)	Upload endpoints, file/CSV validators, target detection, compatibility	FR-008..013; SEC-004
M3 Execution and Metrics (Ayush M Shetty)	Job API, Celery task, sandbox runner, task detection, metrics, status machine	FR-014..021; SEC-005, 006, 010
M4 Benchmark and Reports (Bhavesh Velluru)	Chart data, dashboard, comparison, JSON and PDF export	FR-022..027
Cross-cutting	Security, performance, reliability, usability, portability, retention	NFR-001..011; SEC-001..010

Not tested in Version 1: *model training, streaming inference, deep-learning formats, image/video/NLP data, GPU execution, native mobile clients (all out of scope in Part A, Section 1.4).*

3. Test Strategy

3.1 Test levels and approach

Level	Goal	Approach	Tools	Owner
Unit	Verify functions and classes in isolation	White-box; pytest with mocks for DB, Docker and queue; boundary-value and equivalence partitioning	pytest, pytest-django, coverage.py, Vitest	Each member for own module
Integration	Verify module interfaces, DB, queue, sandbox	API tests against a test PostgreSQL/Redis and real Docker sandbox; contract checks against OpenAPI	pytest + DRF APIClient, Postman/Newman, Docker	Each member; joint in Sprint 5
System (black-box)	Verify end-to-end use cases through the UI	Scripted and manual execution of use-case scenarios; results recorded	Browser, Playwright (smoke subset)	All members
Security	Verify SEC-001..010 and sandbox containment	Authorization matrix, IDOR, injection, token tampering, upload abuse, hostile-model fixtures, dependency and static scans	pytest, OWASP ZAP baseline, Bandit, pip-audit, npm audit, SonarQube	M3 and M1 lead; all review
Performance	Verify NFR-001, 002, 003, 011	Load and timing tests on the Compose deployment	Locust, timing in pytest	M3 and M4
Usability	Verify NFR-005	Task-based walkthrough with five first-time users	Observation sheet	M2 and M4
Regression	Protect existing behaviour	Full unit and integration suite on every pull request; system smoke on merge to main	GitHub Actions	CI

3.2 Automation and continuous integration

- GitHub Actions pipeline on every push and pull request: lint (flake8/ESLint) → unit tests with coverage → build Docker images → integration tests → Bandit/pip-audit/npm audit → SonarQube analysis and quality gate.
- Merge to main is blocked unless tests pass, backend line coverage is at least 80% and at least one reviewer has approved the pull request (NFR-007).
- Test artifacts (JUnit XML, coverage HTML/XML, Newman and ZAP reports) are archived per run as evidence.

3.3 Test data and fixtures

Fixture	Purpose
iris.csv, wine.csv, diabetes.csv (small public tabular datasets)	Classification and regression functional tests; schema mismatch pair (iris model with wine data)
rf_iris.pkl, logreg_wine.joblib, svm_iris.pkl, gbm_housing.pkl, linreg.onnx	Valid scikit-learn and ONNX models trained with the pinned library versions
housing_10k_x_50.csv (10,000 rows, 50 features), 50 MB CSV generator	Performance and size-boundary tests
Corrupt and truncated model files; ZIP renamed to .onnx; file named with path traversal	Negative and validation tests
Hostile-model fixtures: never returns, memory and process exhaustion, tries outbound connection, tries to read paths outside its inputs	Sandbox containment tests; executed only inside the sandbox on the test host

4. Test Environment

Environment	Configuration	Use
Local developer	Docker Compose (backend, worker, Redis, PostgreSQL 15, sandbox image), Python 3.11, Node LTS	Unit and integration tests while developing
CI (GitHub Actions)	Ubuntu runner with Docker; service containers for PostgreSQL and Redis	Automated regression, security scans, coverage
Staging / demo	Docker Compose on a single host mirroring production settings, HTTPS reverse proxy	System, performance, usability and demonstration

5. Entry, Exit and Suspension Criteria

Criterion	Definition
Entry	Requirement baseline approved; module builds in CI; test data and fixtures available; environment reachable.
Exit (per sprint)	All planned test cases for stories in the sprint executed; no open High or Critical defects in those stories; CI green.
Exit (release)	100% of High-priority requirements verified with Pass; at least 90% of all test cases Pass; no open Critical or High defect; backend line coverage >= 80%; SonarQube gate passed; security tests ST-SEC-01..13 all Pass or have an accepted, documented risk.
Suspension	Build or environment broken; sandbox policy test fails (stop all further execution tests until fixed); a Critical security defect is found.
Resumption	Fix merged and verified by the failing test and a regression run.

6. Roles and Responsibilities

Member	Testing responsibility
Atharva Anand	Owns M1 test cases (UT/IT/ST-AUTH); authorization matrix; login rate-limit and token tests
Atharva Rakesh Bidre	Owns M2 test cases (UT/IT/ST-ING); upload abuse and boundary tests; fixtures for files and CSVs
Ayush M Shetty	Owns M3 test cases (UT/IT/ST-EXE); sandbox containment and hostile-model suite; Celery and Docker test harness; performance PERF-02/04
Bhavesh Velluru	Owns M4 test cases (UT/IT/ST-REP); chart and report checks; PDF/JSON validation; Locust API load test PERF-01
All members	Cross-review of each other's pull requests and test cases; execute system tests; record Actual Result and Pass/Fail in the shared test sheet

7. Schedule

Sprint	Weeks	Testing activities
1	1–2	Test framework, CI skeleton, M1 unit and integration tests
2	3–4	M2 unit/integration tests, file fixtures, upload negative tests
3	5–6	M3 unit tests, sandbox integration tests, hostile-model suite, coverage baseline
4	7–8	M4 unit/integration tests, PDF/JSON validation, comparison guard tests
5	9–10	Full integration, system tests through UI, SonarQube review, ZAP baseline, performance tests
6	11–12	Regression, security hardening verification, usability walkthrough, final execution of all cases, validation report

8. Risks and Mitigation

Risk	Impact	Mitigation
Sandbox escape or policy gap	Critical	Dedicated containment test suite run in CI; socket proxy; policy checked by inspecting the container configuration (IT-EXE-04)
Model/scikit-learn version mismatch causes false failures	Medium	Pin versions in sandbox image; clear FAILED message; fixtures built with pinned versions
Docker-in-CI flakiness	Medium	Retry policy for container start; separate marker for Docker tests; local reproduction with Compose
Unequal test effort across modules	Medium	Minimum test-case count per module (this plan); coverage per module reported in Sprint reviews
Time pressure in final sprints	High	Continuous testing from Sprint 1; regression automated; system tests scripted early

9. Test Cases

Each functional feature area has at least 14 test cases (unit, integration and system), above the guideline minimum of 8–10 test cases per implemented use case or functional requirement group; every functional requirement is covered by at least one case (Section 13). ***Actual Result*** *and* ***Result (Pass/Fail)*** *are filled in during manual or automated execution and recorded in the shared test sheet (test-cases.xlsx) and in the final validation report. Type is encoded in the ID: UT = unit, IT = integration, ST = system, PERF/REL/USA/PORT/RET = non-functional.*

9.1 M1 – Authentication and Workspace Management (16 cases)

Test ID	Req.	Description	Steps / test data	Expected result	Actual	Result
UT-AUTH-01	FR-001	Password policy rejects weak password	Call validator with 'abc123' (6 chars)	Validation error: minimum 8 characters		Pass / Fail
UT-AUTH-02	FR-001	Password policy accepts strong password	Call validator with 'Str0ngPass1'	Accepted		Pass / Fail
IT-AUTH-01	FR-001	Register with new email	POST /auth/register {email: a@pes.edu, password: Str0ngPass1}	201; user row created; stored hash differs from plaintext		Pass / Fail
IT-AUTH-02	FR-001	Register with duplicate email	Repeat IT-AUTH-01 with the same email	400/409; no second row; message does not reveal other account data		Pass / Fail
IT-AUTH-03	FR-002	Login with valid credentials	POST /auth/login with registered user	200; access + refresh tokens; role claim = USER		Pass / Fail
IT-AUTH-04	FR-002	Login with wrong password	POST /auth/login with wrong password	401 generic 'invalid credentials'		Pass / Fail
IT-AUTH-05	FR-003	Refresh token rotation	POST /auth/token/refresh with valid refresh token	200 new access token; old refresh token no longer valid		Pass / Fail
IT-AUTH-06	FR-003	Logout revokes refresh token	POST /auth/logout, then reuse refresh token	Second use returns 401		Pass / Fail
IT-AUTH-07	FR-005	History lists only own evaluations	User A has 3, user B has 2 evaluations; GET /evaluations as A	3 items, newest first, page size <= 20		Pass / Fail
IT-AUTH-08	FR-005	History status filter	GET /evaluations?status=FAILED	Only FAILED evaluations returned		Pass / Fail
IT-AUTH-09	FR-006	Delete own evaluation	DELETE /evaluations/{id} as owner	204; row, artifacts and reports removed		Pass / Fail
IT-AUTH-10	FR-006	Delete another user's evaluation	DELETE /evaluations/{id} as non-owner	404 (no existence leak); data unchanged		Pass / Fail
IT-AUTH-11	FR-007	Evaluator shares a course workspace	Evaluator creates workspace, adds student by email; student evaluates in it; evaluator opens list	Evaluator sees the student's evaluation (200)		Pass / Fail
IT-AUTH-12	FR-004, FR-007	USER cannot manage workspaces	USER token calls POST /workspaces	403 Forbidden		Pass / Fail
ST-AUTH-01	FR-001–003	Register, login, logout through the UI	Open app, register, log in, log out, try /history	Redirected to login after logout; no protected data shown		Pass / Fail
ST-AUTH-02	FR-005–007	History page in the UI	Run two evaluations, open History, apply filter, delete one	List, filter and deletion behave as specified		Pass / Fail

9.2 M2 – Dataset Ingestion and Schema Pre-Validation (15 cases)

Test ID	Req.	Description	Steps / test data	Expected result	Actual	Result
UT-ING-01	FR-008	Model extension allow-list	Validate names model.pkl, model.joblib, model.onnx, model.exe, model.pkl.exe	First three accepted; last two rejected		Pass / Fail
UT-ING-02	FR-009	Dataset size boundary	Validate CSV of 50 MB and 50 MB + 1 byte	50 MB accepted; +1 byte rejected (413)		Pass / Fail
UT-ING-03	FR-010	Header-only CSV	CSV with header row and zero data rows	Rejected: 'no data rows'		Pass / Fail
UT-ING-04	FR-010	Duplicate column names	CSV header 'a,b,a,target'	Rejected, duplicate column 'a' named		Pass / Fail
UT-ING-05	FR-011	Target suggestion	CSV with last column 'species' (3 unique strings)	Suggested target = 'species'; user can override		Pass / Fail
UT-ING-06	FR-012	Feature count mismatch	Model expects 4 features; CSV has 3 feature columns	Incompatible; message 'expected 4, got 3'		Pass / Fail
UT-ING-07	FR-010	Missing values in features	CSV with 2 empty cells in column 'age'	Rejected, column 'age' with count 2 reported		Pass / Fail
IT-ING-01	FR-008	Upload valid model	POST /models with rf_iris.pkl	201; stored under UUID name; sha256 and size recorded		Pass / Fail
IT-ING-02	FR-008	Extension/content mismatch	Upload a ZIP archive renamed to model.onnx	400; rejected by signature check; nothing stored		Pass / Fail
IT-ING-03	FR-009	Upload valid dataset	POST /datasets with iris.csv	201; rows = 150, columns listed, dtypes inferred		Pass / Fail
IT-ING-04	FR-012	Compatible pair	POST /validations rf_iris.pkl + iris.csv, target=species	status PASSED; feature list returned		Pass / Fail
IT-ING-05	FR-012, FR-013	Incompatible pair	POST /validations rf_iris.pkl + wine.csv	status FAILED; report names each failed check with expected vs actual		Pass / Fail
IT-ING-06	FR-011	Target override	PATCH /datasets/{id}/target {column: 'petal_width'}	200; target persisted; excluded from feature list		Pass / Fail
ST-ING-01	FR-008–013	Upload and validate in the UI	Drag-drop model and CSV, pick target, click Validate	Progress shown; PASSED/FAILED summary with details		Pass / Fail
ST-ING-02	FR-008, FR-009	Unauthenticated upload	POST /models and /datasets without token	401; no file written		Pass / Fail

9.3 M3 – Sandboxed Model Execution and Metrics Engine (15 cases)

Test ID	Req.	Description	Steps / test data	Expected result	Actual	Result
UT-EXE-01	FR-015	Regression detection	Target: 200 distinct floats	task_type = REGRESSION		Pass / Fail
UT-EXE-02	FR-015	Classification detection	Target: strings 'a','b','c'	task_type = CLASSIFICATION; override honoured		Pass / Fail
UT-EXE-03	FR-017	Classification metrics	y_true=[0,1,1,0], y_pred=[0,1,0,0]	accuracy 0.75; class-1 precision 1.00, recall 0.50, F1 0.667		Pass / Fail
UT-EXE-04	FR-018	Regression metrics	y=[3,5,7], y_hat=[2,5,8]	MAE 0.667, MSE 0.667, RMSE 0.816, R-squared 0.75		Pass / Fail
UT-EXE-05	FR-019	Illegal state transition	Transition SUCCEEDED -> RUNNING	Raises error; status unchanged		Pass / Fail
UT-EXE-06	FR-017	ROC-AUC availability	Binary model with and without predict_proba	AUC reported with proba; omitted with note otherwise		Pass / Fail
IT-EXE-01	FR-014	Create job asynchronously	POST /evaluations with validated pair	202 within 1 s; status QUEUED; Celery task enqueued		Pass / Fail
IT-EXE-02	FR-016	scikit-learn model end to end	Run rf_iris.pkl on iris.csv	SUCCEEDED; metrics stored; container removed		Pass / Fail
IT-EXE-03	FR-016	ONNX regression model	Run linreg.onnx on housing_small.csv	SUCCEEDED; MAE, MSE, RMSE, R-squared stored		Pass / Fail
IT-EXE-04	FR-016	Sandbox run policy	Inspect the launched container configuration	network none; read-only rootfs; non-root; caps dropped; memory/CPU/PID limits set		Pass / Fail
IT-EXE-05	FR-019	Timeout enforcement	Run model fixture that never returns	Killed at 120 s; status TIMED_OUT; container removed		Pass / Fail
IT-EXE-06	FR-020	Corrupt model file	Run truncated model.pkl	FAILED; message contains no stack trace or path		Pass / Fail
IT-EXE-07	FR-021	Results retrievable	GET /evaluations/{id} after success	metrics_json and charts_json present and equal to stored values		Pass / Fail
ST-EXE-01	FR-014–021	End-to-end evaluation	Upload, validate, run, open dashboard	Metrics visible; total time within NFR-002		Pass / Fail
ST-EXE-02	FR-016	Hostile model contained	Run fixture that tries to read host paths and open sockets	No host data reachable; no connection; job ends FAILED with audit entry		Pass / Fail

9.4 M4 – Benchmarking, Visual Analytics and Report Export (13 cases)

Test ID	Req.	Description	Steps / test data	Expected result	Actual	Result
UT-REP-01	FR-024	Best metric selection	F1: 0.82, 0.91, 0.77; RMSE: 4.1, 3.2, 5.0	Best F1 = 0.91 (highest); best RMSE = 3.2 (lowest)		Pass / Fail
UT-REP-02	FR-027	Different dataset guard	Two evaluations with different dataset_sha256	Raises ComparisonError		Pass / Fail
UT-REP-03	FR-023	Comparison size limits	Compare 1, 2, 5 and 6 evaluations	1 and 6 rejected; 2 and 5 accepted		Pass / Fail
IT-REP-01	FR-023	Compare three models	POST /comparisons with 3 evaluation ids on one dataset	200; aligned metric table for 3 models		Pass / Fail
IT-REP-02	FR-027	Mixed task types	POST /comparisons with a classification and a regression id	400 with explanatory message		Pass / Fail
IT-REP-03	FR-025	JSON export	GET /evaluations/{id}/report.json	Valid JSON; includes model, dataset, task type, metrics		Pass / Fail
IT-REP-04	FR-026	PDF for one evaluation	GET /evaluations/{id}/report.pdf	application/pdf; opens; metric table and charts present		Pass / Fail
IT-REP-05	FR-026	PDF for a comparison	GET /comparisons/{id}/report.pdf	All compared models listed; best values marked		Pass / Fail
IT-REP-06	FR-026, SEC-008	Export of another user's data	GET report.pdf of another user's evaluation	404/403; no file returned		Pass / Fail
IT-REP-07	FR-022	Chart data shape	GET /evaluations/{id}/charts for 3-class model	Confusion matrix is 3x3; ROC data present when probabilities exist		Pass / Fail
ST-REP-01	FR-022	Classification dashboard	Open dashboard of a classification evaluation	Metric cards, confusion matrix and ROC curve render		Pass / Fail
ST-REP-02	FR-022	Regression dashboard	Open dashboard of a regression evaluation	Predicted-vs-actual and residual plots render		Pass / Fail
ST-REP-03	FR-023–026	Comparison page and downloads	Select 3 evaluations, compare, download PDF and JSON	Best model highlighted; both files download and open		Pass / Fail

9.5 Security tests (13 cases)

Test ID	Req.	Description	Steps / test data	Expected result	Actual	Result
ST-SEC-01	SEC-001	Passwords are hashed	Inspect user table after registration	Value begins with the hasher identifier; no plaintext		Pass / Fail
ST-SEC-02	SEC-002, FR-004	Privilege escalation	USER token calls EVALUATOR and ADMIN endpoints	403 Forbidden for every call		Pass / Fail
ST-SEC-03	SEC-003	Expired token	Call API with token older than 15 minutes	401 Unauthorized		Pass / Fail
ST-SEC-04	SEC-003	Tampered token	Alter role claim in payload, keep signature	401; signature verification fails		Pass / Fail
ST-SEC-05	SEC-004	SQL injection in filters	GET /evaluations?status=' OR '1'='1	400 or empty list; no data leak; no SQL error		Pass / Fail
ST-SEC-06	SEC-004	Path traversal in file name	Upload file named ../../etc/x.pkl	Stored under UUID name inside media root only		Pass / Fail
ST-SEC-07	SEC-005	No network from sandbox	Run fixture that opens an outbound connection	Connection fails; job reported FAILED; audit entry written		Pass / Fail
ST-SEC-08	SEC-005	Resource exhaustion	Run fixtures that exhaust memory and process count	Container killed by limits; host and other jobs unaffected		Pass / Fail
ST-SEC-09	SEC-008, FR-004	IDOR on evaluations	User A requests user B's evaluation, model and report ids	404 for each; no data returned		Pass / Fail
ST-SEC-10	SEC-007	Login rate limiting	Send 6 failed logins within one minute	6th attempt returns 429		Pass / Fail
ST-SEC-11	SEC-004	Oversized upload	Upload 200 MB file	413 rejected early; server memory stable		Pass / Fail
ST-SEC-12	SEC-009	Dependency and config scan	Run pip-audit, npm audit, Bandit and SonarQube in CI	No critical findings; no secrets in repository		Pass / Fail
ST-SEC-13	SEC-006	Audit log	Trigger failed login and a 403	Audit rows created with user, IP, event and time		Pass / Fail

9.6 Non-functional tests (8 cases)

Test ID	Req.	Description	Steps / test data	Expected result	Actual	Result
PERF-01	NFR-001	API latency under load	Locust: 50 users, 5 min, mix of login/history/status	95th percentile <= 500 ms; error rate < 1%		Pass / Fail
PERF-02	NFR-002	Evaluation duration	10,000 x 50 CSV with 20 MB model	Job duration <= 60 s		Pass / Fail
PERF-03	NFR-011	Validation duration	Validate 50 MB CSV	Completes <= 20 s		Pass / Fail
PERF-04	NFR-003	Concurrent jobs	Submit 5 jobs with 3 workers	All finish; each result equals its single-run baseline		Pass / Fail
REL-01	NFR-006	Worker crash recovery	Kill worker during a RUNNING job; restart	Job marked FAILED within 5 min; no orphan container		Pass / Fail
USA-01	NFR-005	First-run walkthrough	Five new users complete first evaluation	At most 5 primary screens; issues logged		Pass / Fail
PORT-01	NFR-008	Fresh install	Clone repository; run docker compose up on two operating systems	Application reachable; smoke test passes		Pass / Fail
RET-01	NFR-010	Retention job	Age artifacts to 31 days; run retention task	Files and DB references removed		Pass / Fail

10. Test Data Matrix

Scenario	Data	Expected
Valid	rf_iris.pkl + iris.csv, target = species	PASSED; evaluation SUCCEEDED; accuracy reported
Valid (regression)	linreg.onnx + housing_small.csv	SUCCEEDED; MAE, MSE, RMSE, R-squared reported
Invalid type	model.exe, notes.txt as dataset	Rejected with reason
Boundary – size	CSV of exactly 50 MB; 50 MB + 1 byte	Accepted; rejected (413)
Boundary – rows	500,000 rows; 500,001 rows	Accepted; rejected
Mismatch	rf_iris.pkl + wine.csv	FAILED, expected 4 features, got 13
Missing values	CSV with empty cells in a feature column	Rejected, column and count listed
Security – authorization	USER token on evaluator/admin endpoint; user A on user B's ids	403; 404
Security – sandbox	Hostile-model fixtures (network, memory, process, never-return)	Contained; FAILED/TIMED_OUT; host unaffected
Failure	Worker killed mid-job	Job FAILED within 5 min; no orphan container

11. Security Testing Plan

Security validation is planned from the start and executed in Sprints 3, 5 and 6. It combines automated scanning with targeted manual tests.

Area	Technique	Tool / evidence
Authentication and session	Token expiry, tampering, replay of rotated refresh tokens, brute-force limits	pytest, Postman; ST-SEC-03, 04, 10
Authorization	Role matrix for every endpoint; object-level (IDOR) tests with two users	pytest matrix; ST-SEC-02, 09
Input handling	SQL injection, path traversal, oversized and disguised uploads, malformed CSV	ZAP baseline, pytest; ST-SEC-05, 06, 11
Sandbox containment	Hostile-model fixtures for network, memory, process, runtime and filesystem; inspection of container configuration	IT-EXE-04, ST-SEC-07, 08, ST-EXE-02
Configuration and dependencies	Secrets scan, HTTPS/CORS review, dependency vulnerability scan, static analysis	Bandit, pip-audit, npm audit, SonarQube; ST-SEC-12
Logging	Audit rows for failed login, 403, sandbox violation, timeout	ST-SEC-13

11.1 Authorization matrix to be executed (expected result)

Endpoint group	Anonymous	USER (own data)	USER (other's data)	EVALUATOR	ADMIN
/auth/register, /auth/login	Allow	Allow	n/a	Allow	Allow
/models, /datasets, /validations, /evaluations (create)	401	Allow	404	Allow	Allow (no content read)
/evaluations/{id}, charts, reports	401	Allow	404	Own + workspace members	403
/comparisons	401	Allow	404	Allow	403
/workspaces	401	403	403	Allow	403
/admin/users/{id}	401	403	403	403	Allow

12. Defect Management

- Defects are logged in Jira with ID, severity, steps to reproduce, expected vs actual result, linked requirement and test case, owner and status (Open → In Progress → Fixed → Retest → Closed).
- **Severity:** Critical – security breach, sandbox escape, data loss, system unusable; High – major feature fails, no workaround; Medium – feature partly fails, workaround exists; Low – cosmetic.
- A defect fix is merged by pull request that references the Jira key and includes a regression test; the failing test case is re-executed and recorded as Retest Pass.

- The defect summary (ID, severity, root cause, fix, retest) is included in the final validation report.

13. Requirement Traceability Matrix

Requirement → architecture → design → code → unit/integration tests → system test. Code paths are the planned locations under *backend/*.

Table 1: Functional requirements traceability

Req ID	Architecture ref	Design ref	Code / file ref	Unit / integration test IDs	System test ID
FR-001	ARCH-AUTH	Class User; API-01	accounts/serializers.py	UT-AUTH-01, UT-AUTH-02, IT-AUTH-01, IT-AUTH-02	ST-AUTH-01
FR-002	ARCH-AUTH	Class User; API-02	accounts/views.py	IT-AUTH-03, IT-AUTH-04	ST-AUTH-01
FR-003	ARCH-AUTH	API-03, API-04	accounts/views.py	IT-AUTH-05, IT-AUTH-06	ST-AUTH-01
FR-004	ARCH-AUTH	RBAC design; permissions	accounts/permissions.py	ST-SEC-02, ST-SEC-09, IT-AUTH-12	ST-SEC-02
FR-005	ARCH-AUTH	API-08; Evaluation	accounts/history.py	IT-AUTH-07, IT-AUTH-08	ST-AUTH-02
FR-006	ARCH-AUTH	API-10; Evaluation	accounts/history.py	IT-AUTH-09, IT-AUTH-10	ST-AUTH-02
FR-007	ARCH-AUTH	Class Workspace; API-21, API-22	accounts/workspaces.py	IT-AUTH-11, IT-AUTH-12	ST-AUTH-02
FR-008	ARCH-ING	Class ValidationService; API-11	ingestion/validators.py	UT-ING-01, IT-ING-01, IT-ING-02	ST-ING-01
FR-009	ARCH-ING	Class Dataset; API-12	ingestion/validators.py	UT-ING-02, IT-ING-03	ST-ING-01
FR-010	ARCH-ING	ValidationService.validateDataset	ingestion/validators.py	UT-ING-03, UT-ING-04, UT-ING-07	ST-ING-01
FR-011	ARCH-ING	Dataset.suggestTarget; API-13	ingestion/target.py	UT-ING-05, IT-ING-06	ST-ING-01
FR-012	ARCH-ING, ARCH-SBX	ValidationService.checkCompatibility; Seq: Upload & Validate	ingestion/compat.py	UT-ING-06, IT-ING-04, IT-ING-05	ST-ING-01
FR-013	ARCH-ING	API-14	ingestion/views.py	IT-ING-05, ST-ING-01	ST-ING-01
FR-014	ARCH-EXEC	Seq: Run Evaluation; API-15	evaluations/views.py, tasks.py	IT-EXE-01	ST-EXE-01
FR-015	ARCH-EXEC	MetricsEngine.detectTask	evaluations/metrics/task_detect.py	UT-EXE-01, UT-EXE-02	ST-EXE-01
FR-016	ARCH-EXEC, ARCH-SBX	SandboxRunner; Deployment diagram	evaluations/sandbox/runner.py	IT-EXE-02, IT-EXE-03, IT-EXE-04, ST-EXE-02	ST-SEC-07, ST-SEC-08
FR-017	ARCH-EXEC	MetricsEngine.classification	evaluations/metrics/classification.py	UT-EXE-03, UT-EXE-06	ST-EXE-01
FR-018	ARCH-EXEC	MetricsEngine.regression	evaluations/metrics/regression.py	UT-EXE-04	ST-EXE-01
FR-019	ARCH-EXEC	State diagram; Evaluation.transition	evaluations/services.py	UT-EXE-05, IT-EXE-05	ST-EXE-01
FR-020	ARCH-EXEC	Error schema	evaluations/tasks.py	IT-EXE-06	ST-EXE-01
FR-021	ARCH-EXEC	Evaluation.metrics_json; API-16	evaluations/views.py	IT-EXE-07	ST-EXE-01
FR-022	ARCH-BENCH, ARCH-UI	API-17; UI-05	benchmarking/charts.py; frontend/Dashboard.tsx	IT-REP-07	ST-REP-01, ST-REP-02
FR-023	ARCH-BENCH	Class ComparisonSet; API-18	benchmarking/comparison.py	UT-REP-03, IT-REP-01	ST-REP-03
FR-024	ARCH-BENCH	ComparisonSet.bestPerMetric	benchmarking/comparison.py	UT-REP-01	ST-REP-03
FR-025	ARCH-BENCH	ReportGenerator.toJSON; API-19	benchmarking/exporters.py	IT-REP-03	ST-REP-03
FR-026	ARCH-BENCH	ReportGenerator.toPDF; API-20	benchmarking/reports/pdf.py	IT-REP-04, IT-REP-05, IT-REP-06	ST-REP-03
FR-027	ARCH-BENCH	ComparisonSet.assertComparable	benchmarking/comparison.py	UT-REP-02, IT-REP-02	ST-REP-03

Table 2: Security requirements traceability

Req ID	Verified by test IDs
SEC-001	ST-SEC-01
SEC-002	ST-SEC-02
SEC-003	ST-SEC-03, ST-SEC-04
SEC-004	ST-SEC-05, ST-SEC-06, ST-SEC-11
SEC-005	IT-EXE-04, ST-SEC-07, ST-SEC-08, ST-EXE-02
SEC-006	ST-SEC-13
SEC-007	ST-SEC-10
SEC-008	ST-SEC-09
SEC-009	ST-SEC-12
SEC-010	IT-EXE-06

Table 3: Non-functional requirements traceability

Req ID	Category	Verified by
NFR-001	Performance	PERF-01
NFR-002	Performance	PERF-02
NFR-003	Scalability	PERF-04
NFR-004	Availability	Ops review
NFR-005	Usability	USA-01
NFR-006	Reliability	REL-01
NFR-007	Maintainability	CI gate
NFR-008	Portability	PORT-01
NFR-009	Testability	CI gate
NFR-010	Data retention	RET-01
NFR-011	Performance	PERF-03

14. Test Deliverables and Reporting

Deliverable	Content	Produced when
Test plan (this Part C)	Strategy, cases, security plan, traceability	Now; updated each sprint
test-cases.xlsx	All cases with Actual Result and Pass/Fail	Filled at each execution cycle
Unit test report and coverage report	pytest JUnit XML, coverage line and branch %, per module	Every CI run; final copy for submission
Security validation evidence	ZAP, Bandit, pip-audit, SonarQube reports; authorization matrix results; sandbox suite log	Sprints 3, 5, 6
Performance results	Locust report and timing logs against NFR targets	Sprint 5 and final
Defect summary and validation report	Defects, root cause, retest, requirement-level pass/fail, maintenance plan inputs	Sprint 6

— End of document —

Appendix C - Product evidence

Historical /results captures from the TSMP-21 browser validation.

New React workflow captures and recording remain part of final rehearsal.

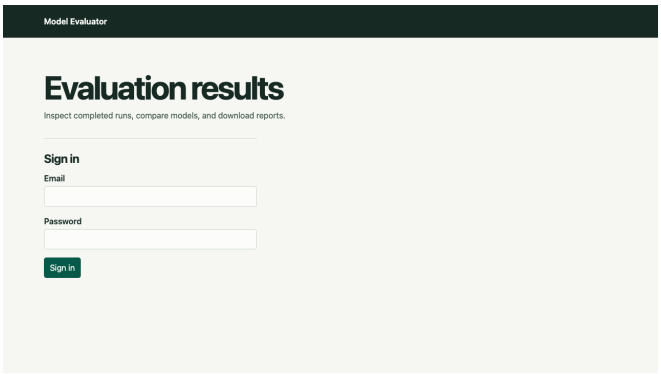


Figure 1. Results dashboard sign-in.

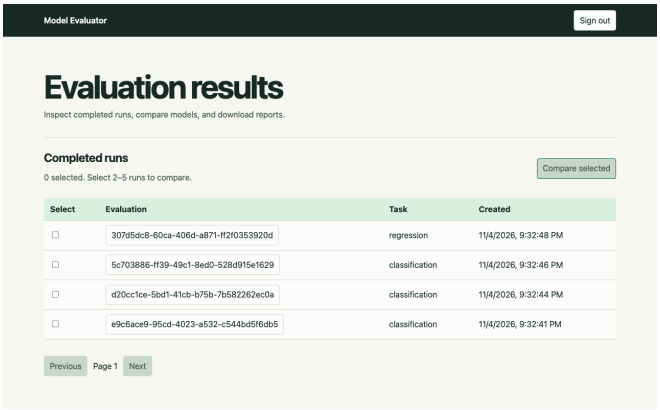


Figure 2. Completed evaluation history.

Appendix C - Product evidence

Historical /results captures from the TSMP-21 browser validation.

New React workflow captures and recording remain part of final rehearsal.

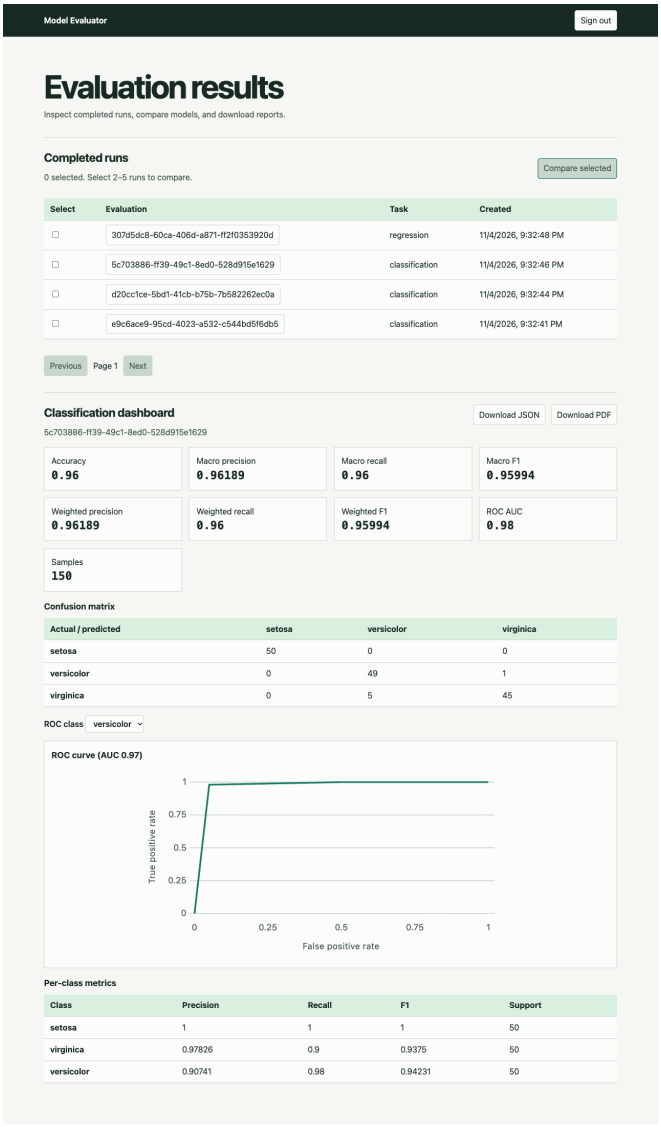


Figure 3. Classification metrics, confusion matrix and ROC.

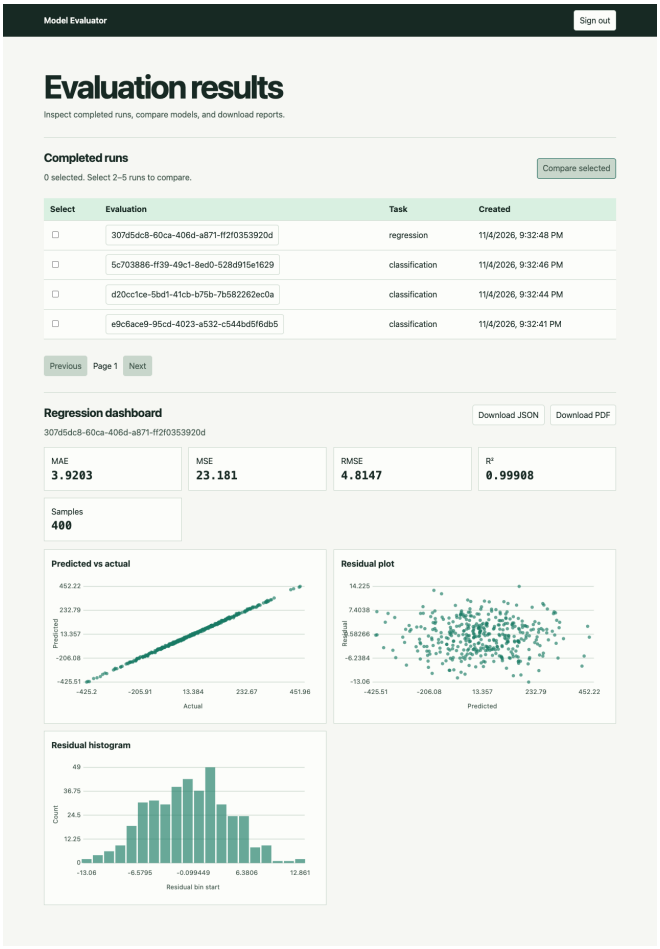


Figure 4. Regression metrics and residual plots.

Appendix C - Product evidence

Historical /results captures from the TSMP-21 browser validation.

New React workflow captures and recording remain part of final rehearsal.

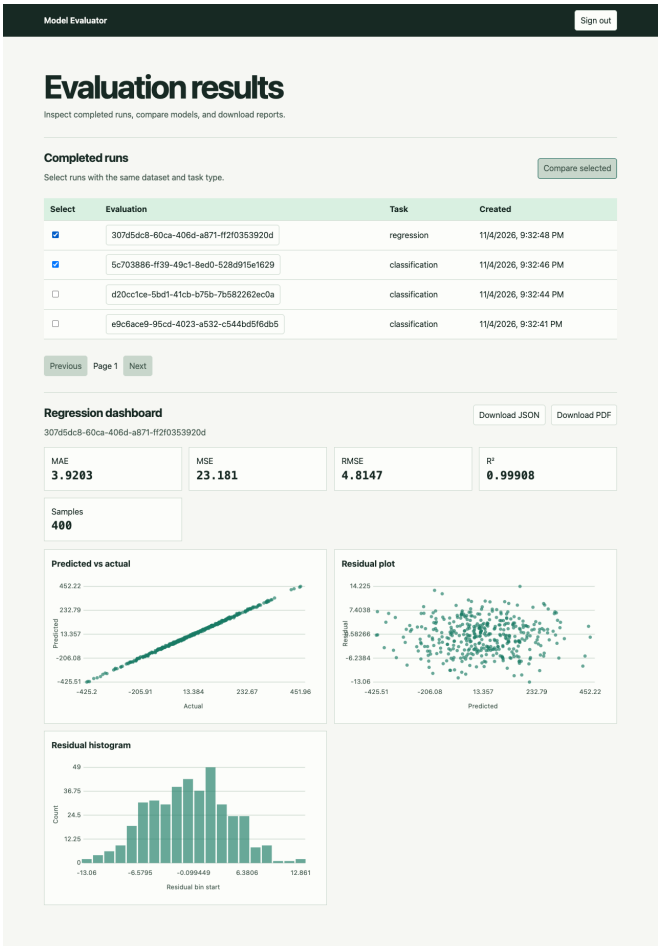


Figure 5. Incompatible comparison selection is blocked.

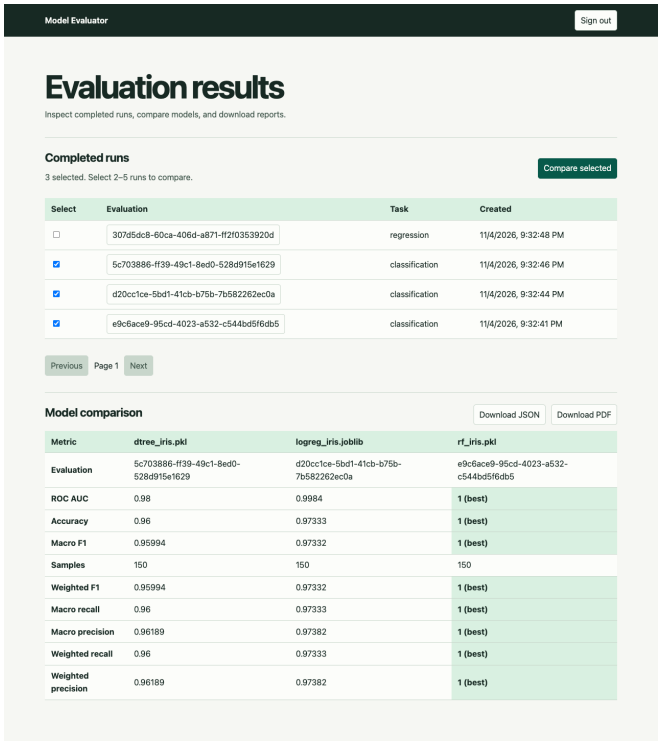


Figure 6. Compatible runs with metric winners.

Appendix C - Product evidence

Historical /results captures from the TSMP-21 browser validation.
New React workflow captures and recording remain part of final rehearsal.

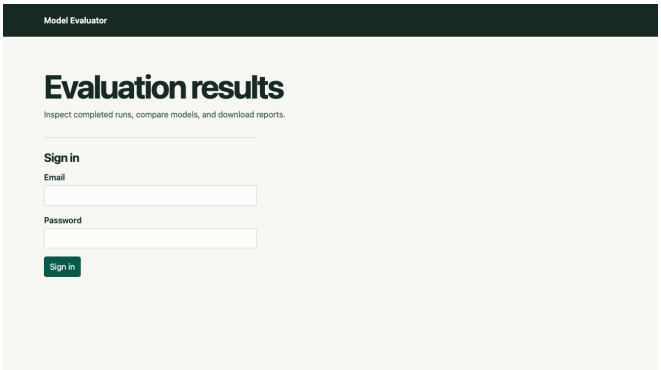


Figure 7. Results dashboard after sign-out.